

Compositional Durable Orchestration for Production Retrieval-Augmented Generation

**A Denotational and Operational Semantics for Change-Driven Indexing,
Evaluation, and Publication in TTC and GEC**

Technical design and executable reference specification

7 August 2026

Abstract

Retrieval-augmented generation systems are often introduced as query-time pipelines, but their operational reliability depends on an asynchronous data plane that continuously converts changing source data into verified, published retrieval artifacts. That data plane must survive process termination, database outages, duplicate delivery, provider throttling, concurrent builds, ambiguous external effects, evaluation regressions, and operator intervention. It must also preserve a precise account of what was built, from which source revision and configuration, at what cost, by which attempts, and under which quality policy.

This report develops a queue-neutral job system for two supplied RAG applications, TTC and GEC/CoinVault, with `ragkit` as their shared semantic boundary. The central design separates the denotation of a workflow from its physical delivery mechanism. Plans are finite, versioned, typed dependency graphs. Sequential composition adds causal edges; tensor composition forms independent parallel branches. Atomic jobs are modeled as graded Kleisli arrows over a state, failure, trace, and cost computation. A structural operational semantics defines run creation, readiness, fenced claims, heartbeats, success, semantic reuse, retry, lease recovery, cancellation, failure propagation, and finalization. Safety properties are stated with proof sketches, including dependency safety, stale-worker exclusion, terminal monotonicity, cache functionality, conservative budget safety, and linearizable alias publication under compare-and-swap.

The implementation is a standalone Go module named `ragjobs`. It includes a canonical plan algebra, an executable transactional in-memory Store, worker interpretation, deterministic backoff, leases and fencing tokens, semantic result identities, content-addressed artifacts, monotone publication aliases, deterministic contiguous change coalescing, indexing and sharded evaluation plans, PostgreSQL control-plane schemas, MySQL and PostgreSQL outbox contracts, River integration guidance, and TTC/GEC adoption mappings. The reference implementation has no external Go dependencies and was validated with formatting checks, static analysis, ordinary tests, race-enabled tests, and an end-to-end demonstration containing an intentional provider retry.

The result does not claim that arbitrary side effects become exactly once. It instead derives observable idempotence from immutable artifacts, semantic keys, transactional inbox/outbox patterns, provider idempotency keys, fenced leases, monotone revisions, and compare-and-swap publication. Production deployment may use either a direct PostgreSQL interpreter or River as the physical queue, but exactly one layer must own physical retry timing. The report closes with a staged deployment plan, operating model, capacity equations, verification evidence, explicit limitations, and a path toward mechanized verification and segment-level incremental indexing.

Table of Contents

Document status and claims.....	11
Contributions.....	11
Reading guide.....	11
1. Introduction.....	12
1.1 The hidden production system behind RAG.....	12
1.2 Research question.....	12
1.3 Design thesis.....	12
1.4 Why category theory is useful here.....	13
1.5 Scope.....	13
2. Supplied codebase and integration boundary.....	14
2.1 Repository topology.....	14
2.2 Architectural seam.....	14
2.3 Why not embed the scheduler in TTC or GEC.....	15
2.4 Existing strengths preserved.....	15
3. Requirements, assumptions, and failure model.....	16
3.1 Functional requirements.....	16
3.2 Non-functional requirements.....	16
3.3 Failure taxonomy.....	16
3.4 Crash points.....	17
3.5 Trust and consistency assumptions.....	17
3.6 Explicit non-goals.....	17
4. Architecture and principal decisions.....	18
4.1 Queue-neutral semantic kernel.....	18
4.2 Plans as data.....	18
4.3 Immutable data plane, mutable control plane.....	18
4.4 Immutable micro-batches before segment mutation.....	19
4.5 Reference indexing graph.....	19
4.6 Decision record.....	19
5. Mathematical foundations.....	21
5.1 Artifact domains.....	21
5.2 Effects and grades.....	21
5.3 Computation domain.....	21
5.4 Parallel composition.....	22
5.5 Open dependency graphs.....	22

5.6 Typed boundaries.....	23
5.7 Partial orders and event structures.....	23
6. Static plan calculus.....	25
6.1 Syntax.....	25
6.2 Node protocol identity.....	25
6.3 Retry policy as plan data.....	25
6.4 Cache mode as semantic assertion.....	25
6.5 Failure modes.....	26
6.6 Canonical plan identity.....	26
6.7 Algebraic laws.....	26
7. Denotational semantics.....	28
7.1 Atomic handler denotation.....	28
7.2 Semantic invocation key.....	28
7.3 Cache denotation.....	28
7.4 Plan denotation.....	29
7.5 Quality gates.....	29
7.6 Publication denotation.....	29
8. Structural operational semantics.....	31
8.1 Configuration.....	31
8.2 Run and node states.....	31
8.3 CREATE.....	32
8.4 CLAIM.....	32
8.5 HEARTBEAT.....	33
8.6 CACHE-HIT and SUCCEED.....	33
8.7 RETRY and FAIL.....	33
8.8 RECOVER.....	34
8.9 CANCEL.....	34
8.10 READY, SKIP, and FINALIZE.....	34
9. Refinement and correctness argument.....	35
9.1 Abstraction relation.....	35
9.2 Internal steps.....	35
9.3 Linearization points.....	35
9.4 Database interpreter obligation.....	35
10. Safety and liveness properties.....	36
10.1 Dependency safety.....	36
10.2 Fence monotonicity and stale-worker exclusion.....	36

10.3 Terminal monotonicity.....	36
10.4 Single accepted outcome per attempt.....	36
10.5 Event-prefix integrity.....	36
10.6 Cache functionality.....	37
10.7 Conservative cost-budget safety.....	37
10.8 Concurrency-key exclusion.....	37
10.9 Alias linearizability and revision monotonicity.....	37
10.10 Trigger deduplication and contiguous adoption.....	37
10.11 Run finalization safety.....	38
10.12 Liveness.....	38
10.13 Exactly-once boundary.....	38
11. Change capture and source consistency.....	40
11.1 The dual-write problem.....	40
11.2 Source revision.....	40
11.3 Relay protocol.....	40
11.4 Contiguous adoption.....	41
11.5 Snapshot protocol for mutable databases.....	42
11.6 Repository snapshots.....	43
11.7 Configuration changes and backfills.....	43
11.8 CDC as an alternative relay.....	43
12. Retry, leases, fencing, cancellation, and budgets.....	44
12.1 Retry layers.....	44
12.2 Deterministic backoff.....	44
12.3 Lease duration.....	44
12.4 Fencing beyond the control database.....	44
12.5 Ambiguous completion.....	45
12.6 Cancellation semantics.....	45
12.7 Supersession.....	45
12.8 Budgets.....	45
12.9 Terminal work and dead letters.....	46
13. Artifact, cache, and publication architecture.....	47
13.1 Content-addressed objects.....	47
13.2 Artifact envelopes.....	47
13.3 Bundle manifest.....	47
13.4 Semantic cache index.....	48
13.5 Publication aliases.....	48

13.6 Publication quality gate.....	48
13.7 Rollback.....	48
13.8 Garbage collection.....	48
13.9 Corruption and quarantine.....	49
14. Production indexing workflow.....	50
14.1 Stage contracts.....	50
14.2 Snapshot.....	50
14.3 Corpus extraction.....	50
14.4 Chunking and representations.....	51
14.5 Lexical branch.....	51
14.6 Dense branch.....	51
14.7 Assemble and verify.....	51
14.8 Evaluate and gate.....	51
14.9 Publish and cleanup.....	52
14.10 Incremental-segment evolution.....	52
15. Evaluation batch semantics.....	53
15.1 Evaluation as a first-class workflow.....	53
15.2 Deterministic partitioning.....	53
15.3 Metrics.....	53
15.4 Stochastic judges.....	53
15.5 Aggregation.....	54
15.6 Baseline comparison.....	54
15.7 Quality policy.....	54
15.8 Experiment linkage.....	54
16. PostgreSQL control-plane interpreter.....	55
16.1 Data model.....	55
16.2 Run creation transaction.....	55
16.3 Claim transaction.....	55
16.4 Completion transaction.....	56
16.5 Failure and recovery transactions.....	56
16.6 Isolation and constraints.....	56
16.7 Notifications.....	56
16.8 Time.....	56
16.9 Partitioning and retention.....	56
17. River versus a native queue.....	58
17.1 River capabilities relevant to the design.....	58

17.2 Direct Store profile.....	58
17.3 River profile.....	58
17.4 Transaction boundaries in River mode.....	58
17.5 Feature comparison.....	59
17.6 Recommendation.....	59
18. TTC integration.....	60
18.1 Existing TTC flow.....	60
18.2 Handler mapping.....	60
18.3 Triggering.....	60
18.4 Experiments.....	60
18.5 Publication.....	61
19. GEC/CoinVault integration.....	62
19.1 Existing GEC boundaries.....	62
19.2 MySQL outbox.....	62
19.3 Handler mapping.....	62
19.4 K3s and GitOps deployment.....	62
19.5 Backfill and schema migration.....	63
19.6 Missing-package limitation.....	63
20. Observability, operations, and security.....	64
20.1 Event model.....	64
20.2 Metrics.....	64
20.3 Tracing.....	64
20.4 Logs.....	64
20.5 Service-level objectives.....	64
20.6 Alerts.....	65
20.7 Administrative controls.....	65
20.8 Secret management.....	65
20.9 Tenant and corpus isolation.....	65
20.10 Supply chain and handler registration.....	65
20.11 Incident procedures.....	66
21. Performance and capacity planning.....	67
21.1 Workload decomposition.....	67
21.2 Change coalescing.....	67
21.3 Full-rebuild cost model.....	67
21.4 Provider rate limits.....	68
21.5 Queue priorities and fairness.....	68

21.6 Backpressure.....	68
21.7 Artifact storage growth.....	68
21.8 Database load.....	69
21.9 Autoscaling signals.....	69
21.10 Benchmark plan.....	69
22. Reference implementation.....	70
22.1 Module structure.....	70
22.2 Plan types.....	70
22.3 Composition API.....	70
22.4 Store contract.....	71
22.5 Memory Store.....	71
22.6 Worker interpreter.....	71
22.7 Handler registry.....	71
22.8 Artifact implementation.....	72
22.9 Trigger implementation.....	72
22.10 RAG plans.....	72
22.11 Demonstration.....	72
22.12 Database contracts.....	72
23. Verification and validation.....	73
23.1 Validation environment.....	73
23.2 Test coverage by property.....	73
23.3 Evidence limits.....	74
23.4 Required production tests.....	74
23.5 Differential state-machine testing.....	74
23.6 Property and model checking.....	74
24. Deployment and adoption plan.....	76
24.1 Phase 0: protocol stabilization.....	76
24.2 Phase 1: PostgreSQL control service.....	76
24.3 Phase 2: handler extraction.....	76
24.4 Phase 3: source triggers.....	76
24.5 Phase 4: shadow builds.....	76
24.6 Phase 5: evaluation and manual promotion.....	76
24.7 Phase 6: automatic gated promotion.....	76
24.8 Phase 7: optimization.....	77
24.9 Schema migration discipline.....	77
24.10 Backup and disaster recovery.....	77

24.11 Multi-region considerations.....	77
24.12 Production readiness checklist.....	77
25. Limitations and future work.....	78
25.1 No live durable adapter in this environment.....	78
25.2 Port schemas are nominal.....	78
25.3 Formalization is not mechanized.....	78
25.4 External side effects remain application-specific.....	78
25.5 Current effect grade is coarse.....	78
25.6 Snapshot and cache policy needs domain proof.....	78
25.7 Evaluation governance.....	79
25.8 Partial rerun and derived runs.....	79
25.9 Dynamic fan-out.....	79
25.10 Segment-level incremental indexing.....	79
25.11 Priority, fairness, and admission.....	79
25.12 Privacy and deletion.....	79
26. Conclusion.....	80
Appendix A. Core serialized contracts.....	81
A.1 Plan.....	81
A.2 Run input.....	81
A.3 Handler output.....	82
A.4 Events.....	82
Appendix B. Operational transition table.....	83
Appendix C. PostgreSQL transaction pseudocode.....	84
C.1 Completion.....	84
C.2 Failure.....	84
C.3 Monotone alias publication.....	84
Appendix D. Operational runbook.....	85
D.1 Run stuck in running state.....	85
D.2 Node repeatedly retries.....	85
D.3 Semantic cache conflict.....	85
D.4 Outbox gap.....	85
D.5 Candidate rejected by gate.....	85
D.6 Alias conflict or stale revision.....	85
D.7 Bad active bundle.....	85
D.8 Scheduler database outage.....	85
D.9 Provider ambiguous request.....	86

Appendix E. Validation evidence.....	87
Appendix F. Glossary.....	88
Appendix G. Distribution map.....	89
References.....	90

Document status and claims

This document is a doctoral-style technical report, architecture specification, implementation guide, and executable-semantics companion. It is not a claim that a doctoral degree has been awarded, nor is it a claim that the supplied TTC and GEC applications were deployed to a live production environment during this work.

The strongest completed claim is narrower and testable: the accompanying `ragjobs` module is an executable reference implementation of the proposed semantics. Its in-memory Store implements the complete public Store contract transactionally under one process; its worker, artifacts, triggers, plans, and tests compile and execute under the available Go toolchain. The distribution also contains PostgreSQL schemas and transition guidance suitable for a durable interpreter, and an explicit River transport protocol. A live PostgreSQL integration test and a compiled River adapter remain production adoption work because the execution environment supplied no database service or external Go dependency resolution.

The original TTC, `ragkit`, `ragopt`, and GEC modules declare Go 1.26-era toolchains and external dependencies, while the available environment provides Go 1.23.2 without dependency downloads. The original modules were therefore inspected as source but not rebuilt. The supplied GEC archive also references `internal/knowledgebuild` without containing that package. This report identifies that missing boundary rather than presenting an unverifiable patch.

Contributions

The work makes six concrete contributions.

First, it identifies a shared architectural seam. `ragkit` already owns reusable RAG semantics: corpus processing, retrieval evaluation, and immutable content-addressed index bundles. TTC and GEC should consume one durable orchestration subsystem at this boundary rather than grow independent schedulers.

Second, it defines a serializable workflow algebra. Atomic, versioned nodes compose sequentially and in parallel. Canonicalization produces a stable plan identity. Named artifact ports are checked across direct dependencies at plan finalization. Effect grades summarize logical reads, writes, external-call classes, and conservative cost bounds.

Third, it gives both denotational and operational semantics. The denotational model explains what a plan means independently of transport. The operational model gives atomic transition rules that a Store must preserve. The distinction prevents queue-library behavior from becoming the definition of business correctness.

Fourth, it supplies a failure model that does not depend on fictional exactly-once execution. Attempts are at least once. Accepted state transitions are fenced. Deterministic work becomes observably idempotent through semantic keys and content-addressed artifacts. Mutable publication is a small, monotone compare-and-swap operation.

Fifth, it specializes the model to production RAG. The indexing plan snapshots a source, extracts and chunks a corpus, builds lexical and dense branches in parallel, assembles and verifies an immutable bundle, optionally evaluates and gates it, publishes an alias, and cleans temporary state. Evaluation runs support deterministic fan-out and fan-in.

Sixth, it provides a runnable Go reference, database contracts, diagrams, adoption guides, a runbook, and validation evidence. The implementation is intended to be read alongside the formal model: the in-memory Store is an executable specification for a SQL interpreter rather than a toy with different semantics.

Reading guide

Chapters 1-10 establish the problem, architecture, formal semantics, and guarantees. Chapters 11-20 cover triggers, retries, artifacts, indexing, evaluation, PostgreSQL, River, and TTC/GEC adoption. Chapters 21-26 cover capacity, implementation, verification, deployment, limitations, and conclusions. Appendices collect protocols, transition tables, SQL pseudocode, runbooks, evidence, and terminology.

1. Introduction

1.1 The hidden production system behind RAG

A retrieval-augmented generation service is frequently drawn as a request-time sequence: encode the question, retrieve documents, rerank, construct a prompt, and generate an answer. That diagram omits the system that makes retrieval possible. Source data changes continually. Product records, categories, code repositories, curated documents, chunking policies, embedding models, lexical analyzers, evaluation sets, and quality thresholds all evolve. Each change can invalidate some or all retrieval artifacts.

The omitted system is an asynchronous derivation engine. It must convert a source state and a configuration state into a candidate index, establish that the candidate is structurally valid, measure it against a named evaluation suite, decide whether it satisfies policy, and publish it without disrupting readers. That work is expensive and failure-prone. It may run for hours, issue thousands of external embedding or judge requests, create gigabytes of temporary artifacts, and cross process and machine boundaries.

A shell command or goroutine is not a production semantics for this work. After a process crash, the system must know which source revision was being processed, which steps committed, which work may be reused, which external calls are ambiguous, whether a retry is permitted, whether the candidate is safe to publish, and whether another worker has already taken ownership. Without durable answers, operators reconstruct state from logs and directories, and correctness depends on luck.

The required system is not merely a queue. A queue transports units of work. The RAG problem additionally requires a durable plan, causal dependencies, typed artifacts, semantic identities, cost and attempt budgets, quality gates, source revision tracking, publication serialization, audit events, and reconciliation. A mature queue can implement part of the operational substrate, but it should not silently define the denotation of an index build.

1.2 Research question

The main research question is:

How can change-driven RAG indexing and evaluation be represented as a compositional, transport-independent computation whose production execution has explicit safety, liveness, retry, identity, and publication semantics?

This question has several subordinate parts.

1. What are the objects and morphisms of the workflow language?
2. Which forms of sequential and parallel composition are valid, and what laws should they satisfy?
3. How are effects, costs, and concurrency constraints represented without conflating them with values?
4. What is the denotation of an atomic job that can fail, consume resources, emit events, and mutate durable state?
5. What operational transitions are sufficient to execute a finite plan under worker crashes and duplicate delivery?
6. What exactly can be guaranteed when external services do not participate in the scheduler transaction?
7. How should database changes be captured and coalesced without losing or reordering revisions?
8. How should immutable index bundles be evaluated and published so readers observe either the old valid bundle or the new valid bundle, never a half-built state?
9. Which responsibilities belong to River or a PostgreSQL queue, and which must remain in the RAG control plane?

1.3 Design thesis

The design thesis is that production correctness becomes tractable when four ideas are combined.

Plans are immutable semantic values. A plan is a finite, versioned, canonical dependency graph. It can be hashed, audited, tested, and interpreted by multiple transports.

Attempts are ephemeral but transitions are durable. Workers receive time-bounded, fenced capabilities. Worker execution may repeat; accepted completion does not.

Artifacts are immutable; publication is minimal and mutable. Expensive results are content addressed. A small alias row or file advances atomically to a verified digest.

Change capture and publication are ordered. Source mutations enter through a transactional outbox. The scheduler adopts only a contiguous revision prefix. Publication rejects a revision that is not newer than the active revision, while accepting an idempotent replay of the same digest and revision.

These choices move correctness from process lifetime into explicit data and transition invariants.

1.4 Why category theory is useful here

Category theory is not introduced as decoration. It provides a vocabulary for separating the shape of composition from the details of an interpreter. A workflow node is a transformation between artifact domains, but its execution also carries failure, durable state, traces, and resource consumption. Monads model this computational context; Kleisli composition models sequencing; a symmetric monoidal product models independent branches; grades summarize effects and conservative cost; string-diagram intuition matches the indexing graph. Moggi's account of computation and monads provides the core semantic separation ([Moggi 1991](#)), while standard categorical structure supplies composition laws ([Mac Lane 1998](#)).

Category theory alone does not solve crashes or transactions. Operational semantics is therefore equally central. Plotkin-style structural operational semantics provides transition rules ([Plotkin 1981](#)). Event structures and partial orders describe concurrency without committing to one interleaving ([Winskel 1987](#)). Kahn's work motivates deterministic composition where independent processes communicate through stable values ([Kahn 1974](#)). Transaction processing and concurrency-control theory explain how a durable Store realizes atomic transitions ([Gray and Reuter 1993](#); [Bernstein, Hadzilacos, and Goodman 1987](#)).

The report uses the smallest mathematical structure that clarifies implementation obligations. It does not claim that every arbitrary side effect commutes, that every job is deterministic, or that the current implementation has been machine-checked in a proof assistant.

1.5 Scope

The system covers:

- change-triggered and manually triggered indexing;
- TTC workspace or application-corpus builds;
- GEC/CoinVault MySQL-backed knowledge builds;
- immutable lexical, vector, and assembled bundle artifacts;
- deterministic and externally judged evaluation batches;
- fan-out/fan-in execution;
- retry, backoff, lease recovery, cancellation, and budgets;
- semantic result reuse for explicitly cacheable nodes;
- quality-gated publication and rollback by alias;
- audit events and operational queries;
- a direct PostgreSQL or River-backed production profile.

The initial design intentionally chooses immutable micro-batched rebuilds over fine-grained in-place index mutation. Segment-level incremental indexing is discussed as future work after the correctness boundary is established.

2. Supplied codebase and integration boundary

2.1 Repository topology

The supplied archive contains four relevant Go modules: `TTC`, `ragkit`, `ragopt`, and a GEC/CoinVault application snapshot. The modules are not organized around one durable scheduler. Instead, each contains synchronous command paths and reusable RAG operations.

`ragkit` is the strongest shared boundary. Its README describes reusable RAG building blocks, including immutable content-addressed index bundles and evaluation components ([go-go-golems 2026b](#)). The package `rag/indexbundle` computes a content-derived bundle identity, verifies an existing bundle when present, writes a new bundle into temporary storage, synchronizes it, and atomically renames it into place. This already embodies the artifact discipline required by the proposed scheduler.

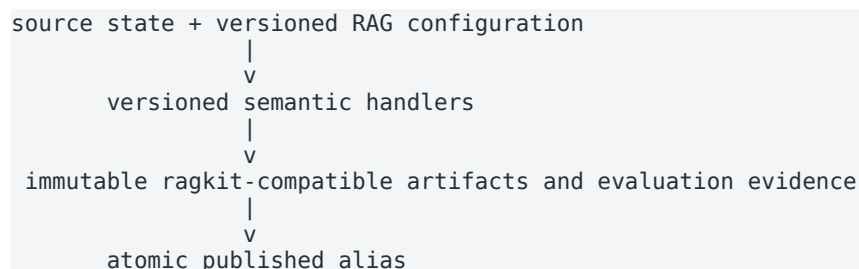
By contrast, `ragkit/flow` explicitly states that it is not a DAG scheduler, does not persist control state, and is not distributed. Its durability is memoization and replay. This is an appropriate inner mechanism for batched embedding or generation, but it cannot answer who owns a multi-hour production build after the process dies.

`TTC`'s workspace index command creates an experiment, prepares a Go workspace corpus, declares embedding resources and retry behavior through its existing flow layer, invokes `indexbundle.Build`, and records completion metrics ([The Tree Center 2026](#)). Its workspace evaluation command provides a natural evaluation-handler boundary. The command contains valuable semantic operations but currently binds them to one process invocation.

GEC/CoinVault exposes `knowledge build`, `knowledge inspect`, and `knowledge eval` commands ([go-go-golems 2026a](#)). `knowledge eval` opens a bundle and evaluates reviewed questions across routes. `knowledge build` invokes `knowledgebuild.Build`, but the supplied archive omits the referenced internal package. The scheduler can still define the exact adapter contract, source consistency protocol, and command boundary; it cannot honestly present a compiled adapter to absent code.

2.2 Architectural seam

The shared seam is therefore:



The scheduler belongs around these operations, not inside retrieval algorithms. It should call stable service methods extracted from `TTC` and `GEC` commands. `ragkit` remains responsible for RAG-specific values and immutable bundles. The orchestration layer is responsible for durable ownership, causal readiness, retries, budgets, audit state, and publication policy.

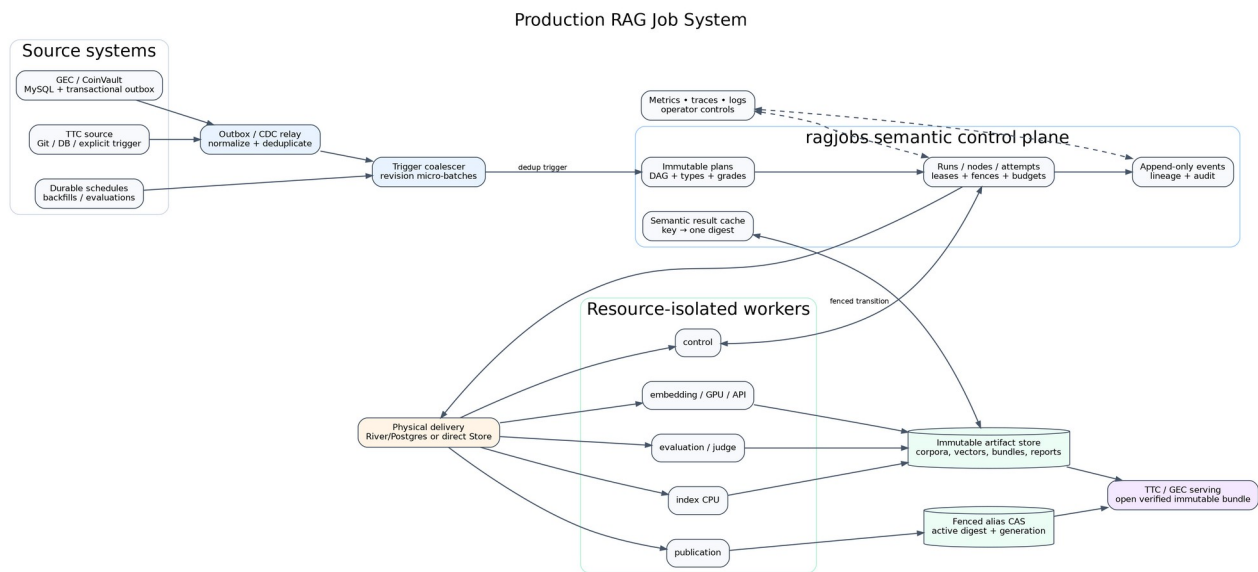


Figure 1. System boundary and principal components. The workflow/control semantics remain independent of direct PostgreSQL or River delivery.

2.3 Why not embed the scheduler in TTC or GEC

Duplicating a scheduler in each application would create divergent semantics for the same problems: source deduplication, retry classes, stale workers, cache identity, evaluation fan-out, and alias publication. It would also make future RAG applications repeat the work.

Placing the scheduler in ragkit itself would also be undesirable. ragkit currently has a focused, reusable data-plane role. A durable scheduler imports operational policy, database schemas, worker lifecycle, queue adapters, retention, and administrative controls. Those concerns should depend on ragkit concepts, not become prerequisites for all ragkit users.

The reference therefore uses a standalone module, ragjobs. TTC and GEC can adopt it incrementally. The module does not import their application packages; adapters register handlers at their command/service boundaries.

2.4 Existing strengths preserved

The proposal deliberately preserves existing strengths rather than replacing them.

- `indexbundle.Build` remains the immutable bundle constructor.
- TTC experiment records remain useful as domain-level evidence linked to a scheduler run.
- `ragkit/flow` remains useful inside an embedding or generation node for request-level batching, repair, caching, and provider retry.
- Existing TTC and GEC evaluation logic becomes the body of evaluation handlers.
- Existing CLIs remain operator and local-development entry points, but they submit or execute the same versioned handlers.

The new system supplies the missing outer control plane.

3. Requirements, assumptions, and failure model

3.1 Functional requirements

A production system must create a run from a manual request, schedule, repository revision, database outbox batch, configuration change, or explicit backfill. It must persist the exact plan and immutable run input. It must expose run, node, attempt, event, artifact, and publication lineage.

A node may execute only after every direct dependency has succeeded. Independent branches should run concurrently. Evaluation should fan out across cases or shards and aggregate deterministically. A run should support fail-fast behavior as well as independent-branch continuation for diagnostic workflows.

Every attempt needs a queue, priority, timeout, retry policy, failure classification, lease, fencing token, worker identity, timestamps, cost accounting, and event trace. Operators need cancellation, inspection, retry or re-run, queue pause/drain at the transport layer, manual promotion, rollback, quarantine, and cache invalidation procedures.

The system must distinguish candidate construction from publication. A failed build or failed quality gate must leave the active index unchanged. A successful publication must be attributable to a run, node, fence, source cursor or revision, plan, and artifact digest.

3.2 Non-functional requirements

The control state must survive worker and scheduler process termination. Duplicate delivery must not corrupt state. A stale worker must not be able to commit a node result after ownership transfers. Database polling must scale across workers without a single global mutex. Idle notification may improve latency but cannot be the sole source of truth.

The plan language and identities must be stable across process runs. Operational metadata such as a trace label or event time must not accidentally invalidate semantic work. Conversely, semantic changes such as handler version, arguments, source cursor, configuration digest, or dependency artifact digest must invalidate reuse.

The system must bound provider and compute spend. Cost bounds must be reserved conservatively before expensive work starts. Attempt and deadline budgets must be enforced durably, not only by cooperative worker code.

Security requirements include secret indirection, least-privilege queue credentials, artifact integrity verification, authenticated administrative actions, auditability, payload size limits, and avoidance of source secrets in plans or events.

3.3 Failure taxonomy

The model classifies failures by the action they permit.

Class	Typical examples	Default disposition
transient	temporary network failure, database failover, object-store timeout	retry with backoff
rate limited	provider quota or HTTP 429-style response	retry, honoring explicit delay
conflict	optimistic lock or alias contention where a fresh read may succeed	retry or reconcile
lease expired	worker death, long pause, lost heartbeat	recover and retry
permanent	malformed data, unsupported schema, missing handler	terminal failure

Class	Typical examples	Default disposition
budget exhausted	cost, attempts, or deadline exceeded	terminal failure
canceled	operator or parent cancellation	terminal cancellation
panic	unexpected handler defect	sanitized terminal failure by default

A failure class is part of the protocol, not a string-matching convention. Adapters should map provider and domain errors explicitly.

3.4 Crash points

Correctness must hold when a process stops at every boundary:

1. before run creation commits;
2. after control rows commit but before a physical queue notification;
3. after a claim commits but before handler execution starts;
4. during an external request;
5. after an external service commits but before the worker observes the response;
6. after an artifact is written but before node completion;
7. after node completion but before the worker receives confirmation;
8. after a dependency succeeds but before children are physically dispatched;
9. during alias publication;
10. after alias publication but before job completion;
11. during cancellation or lease recovery;
12. while an outbox relay copies source events.

The design treats these as normal states to reconcile, not exceptional mysteries.

3.5 Trust and consistency assumptions

The safety results assume that Store methods are atomic and preserve their preconditions. In-memory execution obtains this with one mutex. PostgreSQL execution obtains it with transactions, row locks, unique constraints, and fenced predicates.

The cost safety result assumes handlers report actual cost honestly and do not exceed their declared upper grade. The Store detects an over-grade completion, but it cannot recover money already spent by a dishonest or defective handler.

Semantic cache soundness assumes a cacheable handler is observationally deterministic for its complete semantic input. Provider model aliases, prompts, decoding parameters, data-set revisions, feature flags, locale rules, and code versions must therefore appear in the handler version, arguments, run values, or dependency digests. Nodes whose source state cannot be reproduced or whose evaluator is intentionally stochastic are non-cacheable by default.

Publication safety assumes the alias implementation performs an atomic compare-and-swap and, for ordered database sources, enforces a strictly increasing source revision. Readers must resolve an alias and then open the immutable digest it names; they must not read a directory while it is being mutated.

3.6 Explicit non-goals

The system does not provide arbitrary distributed transactions across MySQL, PostgreSQL, an embedding provider, and object storage. It does not turn non-idempotent external calls into exactly-once calls. It does not infer the semantic identity of opaque application code automatically. It does not replace a domain evaluation methodology. It does not initially maintain a vector index through fine-grained in-place mutation. It does not allow a queue transport to become the sole lineage database.

4. Architecture and principal decisions

4.1 Queue-neutral semantic kernel

The core architectural decision is to separate workflow semantics from queue transport.

The semantic kernel owns:

- immutable plan definitions and plan identities;
- node kinds, versions, arguments, ports, effects, and retry declarations;
- run input and source lineage;
- node/run states and event vocabulary;
- semantic cache identity and conflict detection;
- artifact identities and publication preconditions;
- budgets, quality gates, and application-facing handler protocols.

A transport owns some or all of:

- waking workers;
- selecting ready physical deliveries;
- polling and backoff infrastructure;
- process supervision;
- database connection pooling;
- optional physical job retention and administrative controls.

The direct PostgreSQL profile lets `ragjobs` own both semantic and physical retry state. The River profile lets River own physical delivery and retry timing while `ragjobs` remains authoritative for the plan, semantic state, artifacts, and gates. The same attempt must never be rescheduled independently by both systems.

4.2 Plans as data

A plan is persisted with every run. It is not reconstructed from the latest application code after a crash. Each node identifies a handler by `(kind, version)`, carries canonical JSON arguments, declares direct dependencies and artifact ports, and includes operational policy.

Persisting plans has four consequences.

First, a historical run remains interpretable after the source repository changes. Second, operators can diff plans and explain why semantic reuse did or did not occur. Third, a plan can be validated before any work is claimed. Fourth, multiple interpreters can execute the same plan, including the in-memory executable specification, direct PostgreSQL, and River.

A plan identity is a truncated display form of a SHA-256 digest over canonical plan JSON. The full artifact and semantic digests remain full SHA-256 values. The truncated plan prefix is for usability, not cryptographic authorization.

4.3 Immutable data plane, mutable control plane

The system separates two kinds of state.

The control plane is mutable and transactional: run state, node state, attempts, leases, event sequence, cache index, trigger inbox, and aliases. It is small enough for PostgreSQL.

The data plane is immutable and content addressed: snapshots, normalized corpora, chunk sets, lexical indexes, vectors, assembled bundles, evaluation shards, aggregate reports, and evidence. It belongs in `ragkit` bundle storage, a filesystem, or object storage. Control rows store references and digests rather than large bodies where practical.

This split confines contention. Expensive stages never edit the active index. Publication changes one alias.

4.4 Immutable micro-batches before segment mutation

The first production version should rebuild immutable bundles from a coalesced source snapshot. This choice may perform more work than a perfect incremental indexer, but it dramatically reduces the state space.

An immutable build has one source fingerprint, one configuration fingerprint, one bundle digest, and one verification result. A retry can reuse completed deterministic stages. A failed candidate can remain for diagnosis. Rollback changes an alias. Readers never coordinate with writers beyond alias resolution.

In-place mutation creates harder obligations: exactly which document changes committed, whether lexical and vector indexes cover the same revision, how deletes are tombstoned, when compaction becomes visible, how a crash between index families is repaired, and how evaluation identifies a stable candidate. Those obligations can be met with immutable segments and a manifest in a later phase, but should not be the initial correctness model.

4.5 Reference indexing graph

The reference graph is shown below. Square brackets denote optional stages.

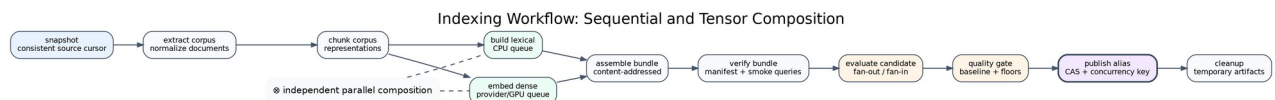


Figure 2. Reference RAG indexing workflow. Lexical construction and dense embedding form a parallel tensor branch; all publication paths terminate in a typed candidate or gate decision.

The graph has the following semantic stages.

`snapshot` establishes a durable, content-addressed source view. For an immutable Git commit or time-travel database snapshot it may be cacheable. For a mutable current-state database snapshot it is non-cacheable by default; its output digest, not the requested cursor alone, identifies the actual snapshot.

`extract-corpus` normalizes source records or repository files. `chunk-corpus` applies versioned chunking and representation policy. `build-lexical` and `embed-dense` consume the same chunk-set digest and may run independently. `assemble-bundle` creates an immutable bundle from both results. `verify-bundle` opens and checks the candidate.

`evaluate-candidate` is optional and is non-cacheable unless the evaluator is explicitly declared deterministic. `quality-gate` is a deterministic policy decision over named evidence. `publish` is never semantically cached and is serialized by a corpus-scoped concurrency key. It consumes the terminal candidate representation: a verified bundle, an evaluation report carrying the candidate reference, or a gate decision carrying the candidate reference. `cleanup` removes only temporary, unreferenced state.

4.6 Decision record

The design can be summarized as a set of binding decisions.

Decision	Consequence
plans are immutable canonical DAG values	historical runs are reproducible and transport-independent
at-least-once attempts with fenced acceptance	crashes are recoverable without pretending execution is exactly once
semantic cache is opt-in	reuse is a correctness assertion, not merely an optimization
immutable artifacts and atomic aliases	build failure cannot partially replace the active index

Decision	Consequence
contiguous source revision adoption	a delayed outbox event cannot be silently skipped
monotone publication revision	an older late run cannot replace a newer active index
exactly one retry authority	River and the control plane cannot multiply retries
PostgreSQL is the production control-state baseline	transactions, row locks, indexes, and SKIP LOCKED support durable multi-worker semantics

5. Mathematical foundations

5.1 Artifact domains

Let $\mathcal{A}$ be a set of artifact schema names. Examples include `rag/snapshot-ref`, `rag/corpus`, `rag/chunk-set`, `rag/lexical-index`, `rag/vector-index`, `rag/index-bundle`, and `rag/evaluation-report`. For each schema $A \in \mathcal{A}$, let $\mathcal{A}(A)$ be the set of canonical values admitted by that schema.

A production value is usually an envelope containing a content digest, media type, logical schema, size, and storage key. The denotation treats this envelope as the value; the bytes it references are immutable and verified against the digest. Small values may be stored directly as canonical JSON. Two values are observationally equal when their canonical representations have the same digest and schema.

Pure deterministic transformations form a category $\mathcal{A}$:

- objects are artifact domains $\mathcal{A}(A)$;
- morphisms are total deterministic functions between validated domains;
- identity is the ordinary identity function;
- composition is ordinary function composition.

Production jobs are not generally pure morphisms in $\mathcal{A}$. They may read external snapshots, write content-addressed objects, fail, consume cost, and emit events. The pure category is therefore the value layer over which a computation structure is built.

5.2 Effects and grades

Let the effect-grade carrier be

$$[G = \{\mathcal{R}\} \{\mathcal{W}\} \{\mathcal{X}\} \{\mathcal{C}\},]$$

where $\mathcal{R}$ is the set of logical read resources, $\mathcal{W}$ the set of logical write resources, $\mathcal{X}$ the set of external-call classes, and $\mathcal{C}$ non-negative cost microunits with a saturating infinity element. A grade is written

$$[g = (g_R, g_W, g_X, g_C).]$$

The grade operation is

$$[gh = (g_R \mathbin{h} g_R, g_W \mathbin{h} g_W, g_X \mathbin{h} g_X, g_C \mathbin{h} g_C),]$$

where $\mathbin{h}$ is saturating addition. The unit is $((_, 0))$. This is a commutative monoid. The implementation's `JoinEffects` realizes the operation by sorted idempotent unions and saturating cost addition.

The grade is an upper summary, not a complete effect system. It supports admission, explanation, policy, and conservative budget reservation. It does not prove that two named external systems commute. A node-local `ConcurrencyKey` is therefore kept outside the composite grade. It is an operational serialization capability attached to the atomic effect that needs it, especially publication.

Graded monads provide the semantic pattern: a computation carries a grade that composes with sequencing ([Katsumata 2014](#); [Orchard, Wadler, and Eades 2020](#)). The implementation uses a practical, first-order grade rather than a type-level proof object.

5.3 Computation domain

Let $\mathcal{W}$ be the set of abstract world states. A world includes the durable control database, immutable artifact namespace, aliases, and abstract states of external services. Let $\mathcal{E}$ be classified errors, $\mathcal{T}$ event traces under concatenation, and $\mathcal{C} = \mathbb{R}_{\geq 0}$ measured cost.

Define an outcome over value domain $\mathcal{A}$ as

$$[(A) = (A \ C) + (E \ C).]$$

To accommodate external nondeterminism and underspecified scheduling, define the graded computation relation

$$[T_g A = _{} \{ \} ((A)),]$$

subject to the obligation that every represented outcome stays within grade (g). A deterministic handler produces a singleton set for each admissible world. A stochastic judge or ambiguous external request may produce several possible outcomes.

The unit computation is

$$[_A(a) = \{(a, 0)\}.]$$

Given $(m \ T_g A)$ and $(f:AT_h B)$, graded bind performs relational composition, threads world state, concatenates traces, adds cost, and propagates failure:

$$[(m \ f) T_{\{gh\}} B.]$$

The ordinary monad laws hold under extensional equality of the outcome relations, trace associativity, and cost associativity. The grade laws follow from the monoid laws of (G). This supplies the semantic basis for versioned atomic nodes as Kleisli arrows

$$[f:AT_g B.]$$

This construction combines state, exceptions, trace, cost, and finite nondeterminism. It is deliberately abstract about how PostgreSQL, a filesystem, or an external provider realizes (). Moggi's separation between values and computations is the key point ([Moggi 1991](#)), not allegiance to one concrete monad transformer stack.

5.4 Parallel composition

For independent computations $(f:AT_g B)$ and $(h:CT_k D)$, a tensor computation has the shape

$$[f \ h : AC \ T_{\{gk\}}(BD).]$$

Operationally, the two branches may be interleaved. Denotationally, their result is schedule-independent only under an independence relation. Let $(f \ h)$ mean that:

1. their mutable writes are disjoint or commute;
2. neither reads mutable state invalidated by the other's write;
3. external effects either commute or are protected by independent idempotency keys;
4. each branch's successful value is deterministic for its semantic input;
5. trace comparison quotients away the order of independent events, or the trace explicitly records the chosen interleaving without changing the artifact result.

When (fh) , adjacent independent steps may be swapped without changing the final artifact/world observation. Repeated adjacent swaps establish equivalence for all linear extensions of the same dependency partial order. When independence is not established, the graph may still contain parallel-ready nodes, but the Store must serialize them through a concurrency key or the handler protocol must reconcile conflicts.

This conditional statement is important. A symmetric monoidal syntax does not make arbitrary external effects commutative. It makes potential independence explicit and gives the implementation a place to enforce the missing condition.

5.5 Open dependency graphs

A raw workflow fragment is a finite labeled directed graph with designated entry and exit node sets:

$$[P = (V, E, I, O).]$$

- (V) is a finite set of stable node identifiers.

- (EVV) is an acyclic dependency relation.
- () maps each node to its versioned specification.
- (IV) is the entry boundary.
- (OV) is the exit boundary.

An atomic node produces a one-vertex fragment. The identity fragment is empty. Sequential composition $(P;Q)$, implemented as $\text{Then}(P, Q)$, takes a disjoint union and adds an edge from every exit of (P) to every entry of (Q) . Tensor composition (PQ) , implemented as $\text{Tensor}(P, Q)$, is disjoint union.

Node identifier disjointness is a composition precondition. Stable identifiers are not incidental display strings; they are names in the audit trail and canonical plan representation.

The fragment construction presents a strict symmetric monoidal category after quotienting raw representations by canonical sorting and restricting to validated acyclic graphs with disjoint names. Sequential associativity follows because both parenthesizations add the same boundary edges. The empty fragment is a left and right identity. Disjoint union is associative, has the empty fragment as unit, and is symmetric after canonical sorting by stable identifier.

The implementation tests these laws directly. This is not a proof that every Go mutation is impossible; it is an executable check that the constructors realize the stated algebra for representative values, supported by the simple definitions.

5.6 Typed boundaries

Each node declares named input and output ports. A port is a pair $((n,A))$ of a local name and artifact schema. The implementation passes all direct dependency outputs to a handler as a map keyed by node identifier. Ports therefore describe and validate the expected artifact interface rather than determine a positional calling convention.

A finalized plan satisfies:

1. every port name and type is non-empty;
2. input names are unique within a node;
3. output names are unique within a node;
4. a node declaring inputs has at least one direct dependency;
5. every declared input type appears among the outputs of a direct dependency;
6. every dependency names an existing node;
7. the dependency graph is acyclic.

Extra dependency outputs may be ignored, and one immutable output may be read by several consumers. This is the ordinary copy/discard behavior expected for artifact references. The current runtime does not yet validate the JSON body against a registry schema; handlers and artifact readers perform domain validation. A schema registry is a stated future extension.

The categorical composition is thus defined on validated fragments. Then constructs edges; `Finalize` rejects a composition whose declared artifact boundary is not supplied. Delaying the type check until finalization allows a fragment to acquire additional dependencies before it is closed into a plan.

5.7 Partial orders and event structures

A plan DAG induces a causality partial order $(_P)$, the reflexive-transitive closure of dependency edges. A valid successful execution trace must respect this order: a node's success precedes readiness of every descendant.

Concurrency is represented by incomparability. Two nodes with neither $(u_P\ v)$ nor $(v_P\ u)$ may be enabled together, subject to resource and concurrency constraints. This resembles an event-structure account in which causality restricts possible configurations and conflict may arise from failure modes or exclusive resources ([Winskel 1987](#)).

The operational event log is a total sequence because a database stores ordered rows. It is one linearization of a richer partial-order execution. Causal reconstruction uses node dependencies and attempt identities, not sequence number alone. Sequence numbers nevertheless provide a stable audit prefix and simplify incremental event consumers.

6. Static plan calculus

6.1 Syntax

A compact abstract syntax is:

```
[  
]
```

Here (n) is a stable node identifier, (k) a handler kind, (v) a handler protocol version, (a) canonical arguments, () typed ports, () retry policy, (g) effect grade, (c) cache mode, and (f) failure mode. Queue, priority, timeout, labels, and concurrency key are persisted operational annotations.

Fragment is the compositional syntax value. Plan is the closed, canonical, validated value. A plan also has a name, plan version, schema version, and labels.

6.2 Node protocol identity

The pair (Kind, Version) names the durable worker protocol. Changing code without changing this pair is permitted only when the change preserves semantics for every persisted input. A change to output shape, normalization, model, prompt, error classification, or side-effect protocol requires a new version or an argument/configuration digest that participates in semantic identity.

A practical versioning rule is:

- patch implementation defects that restore the already documented semantics without changing the version;
- change the version for any observable semantic change;
- keep old handlers available until no nonterminal run or retained replay requires them;
- reject a claim permanently when no compatible handler is registered.

This is analogous to a durable message protocol. A queue payload can outlive a deployment.

6.3 Retry policy as plan data

A retry policy contains maximum attempts, base delay, maximum delay, multiplier, and jitter fraction. Defaults are normalized during finalization. Persisting the policy makes historical behavior explainable and prevents a restart under new application defaults from silently changing an existing run.

The deterministic backoff function is conceptually

```
[ d_a = (d_{}, d_0 m^{a-1}), (1+j), ]
```

where (a) is the failed attempt number and () is derived deterministically from the run identifier, node identifier, and attempt. Deterministic jitter spreads different runs while keeping replay and tests stable. An explicit provider retry delay overrides computed backoff.

Retry policy is subordinate to the run budget. A node can have attempts remaining while the run has exhausted its total-attempt budget, cost budget, or deadline.

6.4 Cache mode as semantic assertion

CacheOff means every claim executes the handler unless a transport-level duplicate is rejected by lease state. CacheContent means the node asserts observational determinism under its semantic key.

This distinction is intentionally severe. A cache mode is not a suggestion that a computation is expensive. It asserts that successful results with the same semantic identity must have the same digest. The Store treats two different digests for one key as a semantic contradiction and rejects the second insertion.

Mutable current-state snapshots and stochastic judge calls are non-cacheable by default. An immutable Git commit snapshot may be cacheable. Embeddings may be cacheable when the exact provider model revision and representation configuration are pinned. Deterministic retrieval metrics may be cacheable; free-running LLM judgments generally are not unless the organization accepts a recorded-result semantics and versions every relevant dependency.

6.5 Failure modes

`FailRun` is the production default. A terminal node failure cancels all remaining nonterminal nodes and finalizes the run as failed. This avoids wasting expensive work after a required stage becomes impossible.

`ContinueRun` allows independent branches to complete. Descendants whose dependency has failed become skipped; unrelated branches continue. The run still finalizes as failed when any node is failed or skipped. This mode is useful for diagnostic suites, multi-arm evaluations, and partial evidence collection.

The distinction is orthogonal to retry. Failure mode applies only after an attempt becomes terminal under its retry and budget conditions.

6.6 Canonical plan identity

`Finalize` performs a pure normalization:

1. clone caller-owned slices and maps;
2. apply queue, cache, failure, and retry defaults;
3. sort dependencies and effect sets;
4. sort nodes by stable identifier;
5. set the plan schema version;
6. validate identifiers, ports, dependencies, retry policy, and acyclicity;
7. clear any supplied plan identifier;
8. hash canonical JSON;
9. derive the display plan identifier.

The canonical JSON encoder recursively sorts object keys and preserves array order. Plan normalization sorts arrays whose order is semantically irrelevant before hashing. Numeric lexical forms are preserved; configuration producers should therefore generate normalized JSON rather than alternate between equivalent forms such as 1 and 1.0.

Proposition 6.1 (Plan identity stability). If two plan values differ only by map-key order, node order, dependency order, effect-set order, or omitted fields replaced by the same defaults, `Finalize` produces the same plan identity.

Proof sketch. Normalization maps each irrelevant ordering and omitted default to one representation. Canonical object encoding maps key permutations to one byte string. SHA-256 is applied to that string. The implementation test constructs reordered plans and checks equal identifiers while also checking that finalization does not mutate the caller.

6.7 Algebraic laws

Let equality of fragments mean equality of their canonical node, dependency, entry, and exit representation.

Proposition 6.2 (Sequential associativity). For pairwise name-disjoint fragments,

$$[(P;Q);R = P;(Q;R).]$$

Proof sketch. Both sides contain the same disjoint union of vertices and internal edges. Both add all edges from exits of (P) to entries of (Q), and from exits of (Q) to entries of (R). Entry and exit boundaries are those of (P) and (R), respectively. Canonical sorting erases construction order.

Proposition 6.3 (Identity). For every fragment (P),

[;P=P=P;.]

This follows directly from the constructor's empty-fragment cases.

Proposition 6.4 (Tensor symmetry and associativity). For pairwise name-disjoint fragments,

[$PQ=QP$, $(PQ)R=P(QR)$.]

Disjoint union has these laws modulo ordering; canonical sorting makes the implementation representation equal.

Proposition 6.5 (Effect homomorphism). `AggregateEffects` maps fragment tensor and sequential composition to the grade monoid operation:

[$(P;Q)=(P)(Q)$, $(PQ)=(P)(Q)$.]

Both constructors contain exactly the union of atomic nodes, and aggregate grades fold the same commutative monoid. The equality gives a conservative whole-plan effect and cost summary, though run-time reservations remain node-local to preserve concurrency.

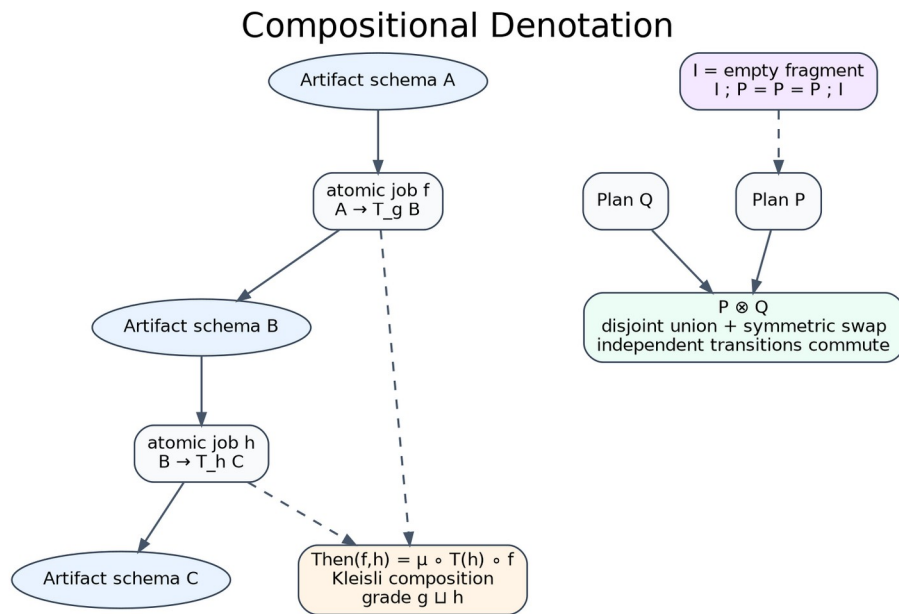


Figure 3. Categorical interpretation of atomic jobs, sequential/Kleisli composition, and tensor parallelism.

7. Denotational semantics

7.1 Atomic handler denotation

A registered handler for $(\text{kind}, \text{version})$ denotes a graded Kleisli arrow. Its semantic input consists of:

- immutable run values;
- semantic trigger fields;
- canonical node arguments;
- direct dependency values and digests;
- a stable source or external-service environment referenced by those values.

Operational fields such as worker ID, attempt number, fence, lease expiration, event occurrence time, and trace labels are not part of the value denotation. They are capabilities and observations of execution.

The handler may use attempt and fence operationally, for example as an idempotency key or conditional-write token, but doing so must not change the artifact value for a cacheable node. A publication handler is non-cacheable precisely because its denotation is a control-state transition, not a reusable artifact function.

7.2 Semantic invocation key

For node (n) , run input (i) , and direct dependency digest map (D) , define

$[K(n,i,D) = H((n), ((n)), (i), (D)).]$

The semantic run projection is

$[i]=(, , ,).]$

It excludes occurrence time, trigger deduplication key, labels, and execution budget. Those fields explain or constrain an execution but do not alter the transformation's value. Dependency entries are sorted by stable node identifier.

The key includes both the source cursor and dependency digests. The cursor prevents accidental reuse across declared source positions. Dependency digests ensure that even when a snapshot step is non-cacheable and observes a later state, all downstream identities are tied to the actual snapshot artifact.

A mutable snapshot node itself must not claim to be a deterministic function of a lower-bound cursor. This is why `IndexOptions.ImmutableSource` controls snapshot caching. For a database current-state build, the snapshot runs and emits a new digest. For a Git commit, the same commit and configuration may safely reuse the snapshot.

7.3 Cache denotation

The semantic cache is a partial function

$[M:K(V,H(V),c,t).]$

A lookup can replace handler execution with the recorded value. An insertion is valid only when the key is absent or the existing digest is equal.

Definition 7.1 (Observationally deterministic node). A node is observationally deterministic when, for every world pair that agrees on all semantic resources named by its complete input, every successful outcome has the same canonical output digest.

A cacheable node asserts this property. Failure timing and trace details may differ. Cost may differ between original execution and reuse; a cache hit records zero current execution cost while preserving source provenance.

Proposition 7.2 (Cache substitution). For an observationally deterministic node, replacing execution with a valid cached result preserves downstream artifact denotation.

Proof sketch. The cached output digest equals the unique successful digest for the semantic key. Downstream semantic identities depend on dependency digest rather than attempt history. Therefore every downstream node receives an observationally equal dependency.

The proposition does not state that a cache hit reproduces the same event trace or cost. Those are operational observations, not the artifact denotation.

7.4 Plan denotation

A closed plan denotes a family of computations indexed by valid topological schedules. Let $((P))$ be the set of linear extensions of the plan partial order. For schedule $(= (v_1, v_n))$, compose node denotations in schedule order, supplying each node with the immutable outputs of its dependencies. Failed outcomes follow the plan failure policy.

The raw denotation is

[$P = \{(P)\} P.$]

For a well-behaved RAG plan, independent successful nodes write content-addressed objects and do not mutate shared names. Under the independence conditions of Section 5.4, all successful schedules produce the same map from node identifiers to output digests and the same terminal publication decision. They may produce different interleavings in the event trace.

Theorem 7.3 (Schedule independence under commutation). Suppose every incomparable pair of enabled nodes in (P) is independent, all cacheable nodes are observationally deterministic, and publication nodes sharing an alias are serialized. Then all successful linear extensions of (P) produce the same terminal artifact map and alias target.

Proof sketch. Any two linear extensions of a finite partial order are connected by a sequence of adjacent swaps of incomparable elements. Independence makes each adjacent swap preserve world and artifact observation. Induction over the swap sequence yields equality. The serialized publication step has one place in the relevant alias order and cannot be swapped with another conflicting publication.

This theorem explains why the artifact discipline matters. Two branches that mutate one shared directory do not satisfy its hypotheses. Two branches that write separate content-addressed objects do.

7.5 Quality gates

A quality gate is a deterministic function

[$q: (E_c, E_b), \text{ }]$

where (E_c) is candidate evidence, (E_b) optional baseline evidence, and $()$ a versioned policy. A decision contains at least the candidate bundle digest, policy digest, metric summary, pass/fail result, and reasons.

The gate should not query mutable policy by an unversioned name during execution. Run values or node arguments must contain a policy digest or immutable version. This makes the decision reproducible and cacheable even when the evaluation shards themselves are non-cacheable.

A gate failure is not a scheduler infrastructure failure. The handler should return a valid decision artifact whose value is `reject`; the publication stage then does not execute, or the gate node returns a classified permanent domain error according to local policy. Returning evidence as a value is generally preferable because it preserves a successful evaluation run with an explicit rejection result.

7.6 Publication denotation

Publication is a conditional state transition over an alias:

[$(a, d, e, r): () +, \text{ }]$

where (a) is alias name, (d) candidate digest, (e) expected current digest or generation, and (r) optional ordered source revision.

For a current alias ((d₀,g₀,r₀)):

1. if ((d,r)=(d₀,r₀)), return the current alias as an idempotent replay;
2. if (r>0), (r₀>0), and (rr₀), reject as stale or conflicting;
3. if the expected digest/generation does not match, reject as a compare-and-swap conflict;
4. otherwise write ((d,g₀+1,r)) atomically with provenance.

The same-digest same-revision case handles an ambiguous outcome in which alias publication committed but node completion did not. The monotone revision check prevents a late outbox event or slow older build from replacing a newer database-derived index.

8. Structural operational semantics

8.1 Configuration

An operational configuration is

[C=(R,N,A,E,M,L,B,Q,t),]

where:

- (R) maps run identifiers to persisted run records;
- (N) maps run/node pairs to node records;
- (A) is attempt history;
- (E) is the append-only per-run event trace;
- (M) is the semantic cache;
- (L) is the set of live leases represented inside node records;
- (B) is consumed and reserved budget state;
- (Q) is the durable set of available nodes or corresponding physical dispatches;
- (t) is authoritative logical time.

A Store method is one atomic transition (CC') or a read. In a PostgreSQL interpreter, authoritative time should be database time or values derived consistently inside the transaction. Application clocks are suitable for tests but should not decide lease ownership across machines.

8.2 Run and node states

Runs have states:

```

queued -> running -> succeeded
              -> failed
              -> canceled

```

Nodes have states:

```

blocked -> available -> running -> succeeded
                                   \-> available (retry)
                                   \-> failed
blocked -----> skipped
blocked/available/running -----> canceled

```

Terminal run states are succeeded, failed, and canceled. Terminal node states are succeeded, failed, skipped, and canceled. Terminal states are monotone.

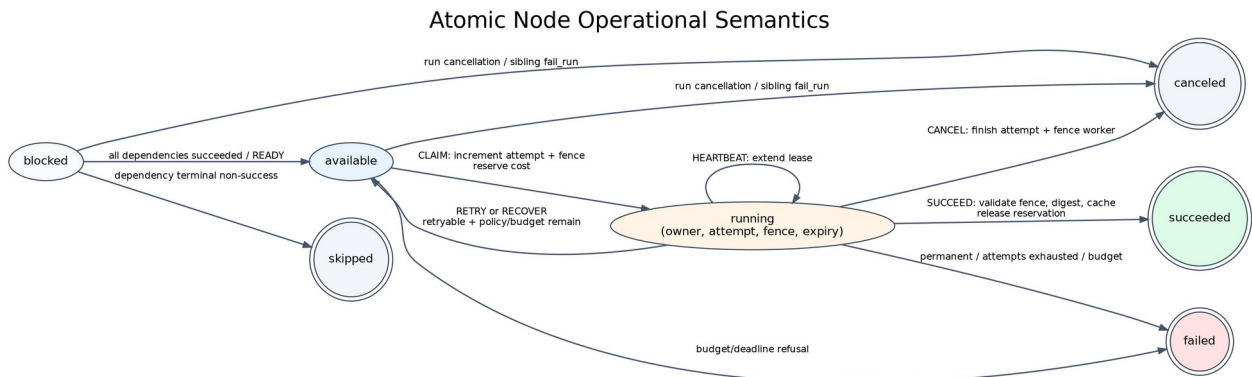


Figure 4. Run/node operational state machine. Retry returns a failed attempt to node availability; lease recovery follows the same retry decision under a new fence.

8.3 CREATE

Given a finalized valid plan (P), immutable input (i), unique run identifier (u), and current time (t), CREATE inserts:

- one run in queued state;
- one node record per plan node;
- roots in available state;
- non-roots in blocked state;
- a run_created event;
- one node_ready event per root.

If the trigger carries a deduplication key, the durable uniqueness domain is

[(„).]

A duplicate returns the existing run identifier and performs no second creation transition. Including the plan identity permits an intentional new plan to process the same source batch while suppressing accidental replay of the same plan.

8.4 CLAIM

A node is claimable when:

- its state is available;
- available_at <= t;
- its run is nonterminal;
- its queue is accepted by the worker;
- the run deadline is open;
- total-attempt budget remains;
- its declared maximum cost can be reserved;
- its concurrency key is not active.

The transition is summarized by the inference rule

[.]

CLAIM atomically:

1. moves the run from queued to running if necessary;
2. increments node attempt count;
3. increments node fencing token;
4. sets owner and lease expiration;
5. reserves the node's declared maximum cost;
6. increments run total attempts;
7. inserts an attempt row;
8. appends run_started when appropriate and node_started;
9. returns a lease containing immutable direct dependency outputs.

The lease is a capability

[=(u,v,a,f,w,e),]

with run, node, attempt, fence, worker, and expiration. Every mutation of a running node must present the current capability.

8.5 HEARTBEAT

A heartbeat is valid only if the run is nonterminal, node is running, owner/attempt/fence match, and the current lease has not expired. It moves expiration forward and appends a heartbeat event.

A worker may continue executing after a heartbeat failure because cancellation is cooperative and the external call may not stop. Such a worker has lost authority. Its later completion is rejected. Handlers with externally visible effects must therefore carry the fence or an idempotency/reconciliation key into the effect protocol; Store fencing alone protects Store state, not an unrelated provider.

8.6 CACHE-HIT and SUCCEED

Before executing a cacheable node, the worker computes its semantic key and queries (M). A hit submits a normal fenced completion marked reused; the Store, not the worker, still decides whether the lease is current.

A success transition requires:

- current matching lease;
- completion time before lease expiration;
- valid JSON output;
- recomputed digest equal to supplied digest;
- non-negative actual cost no greater than declared maximum when bounded;
- no cache row with the same key and a different digest.

The Store then:

1. finishes the current attempt;
2. releases reserved cost;
3. adds actual cost;
4. persists output, digest, semantic key, and reuse flag;
5. clears the lease;
6. inserts the cache value if absent;
7. appends success or reuse event;
8. activates newly unblocked children;
9. reaches a propagation fixed point for skipped descendants;
10. finalizes the run if every node is terminal.

Completion is accepted at most once for a fence. A repeated completion after commit observes a non-running node and returns lease lost. This is safe even when the worker did not receive the original acknowledgment.

8.7 RETRY and FAIL

A failure submission finishes the attempt, releases reservation, adds measured cost, records class/message, and clears the lease. It returns the node to available at `retry_at` exactly when all of the following hold:

- the class is retryable;
- node attempts remain;
- total run attempts remain;
- deadline remains open;
- cost budget has not been exhausted.

Otherwise it marks the node failed. Under `FailRun`, all remaining nonterminal nodes are canceled and the run fails. Under `ContinueRun`, causal descendants eventually become skipped while independent branches continue.

A panic is recovered by the worker and sanitized. The reference maps panic to permanent failure by default because repeated code defects rarely benefit from immediate automated retry. An adapter may map a narrowly understood panic class differently, but should not retry arbitrary panics indefinitely.

8.8 RECOVER

The lease reaper selects running nodes whose lease expiration is not after current time. For each, it finishes the attempt with lease-expired classification and applies the same retry/terminal decision as an ordinary failure. A recovered retry becomes available and receives a strictly greater fence on its next claim.

Recovery must not simply clear the owner field. Doing so would lose attempt history, event causality, cost reservation, and retry accounting. The same transition logic keeps the executable specification and SQL interpreter aligned.

8.9 CANCEL

Run cancellation atomically marks all nonterminal nodes canceled, finishes any live attempts, releases reservations, clears leases, records the reason, and marks the run canceled. Every old lease is thereby fenced because the run is terminal and nodes are no longer running.

Transport cancellation is additional. In River mode the control plane requests River job cancellation and workers honor context cancellation. The durable control transition remains authoritative if a worker cannot be interrupted immediately.

8.10 READY, SKIP, and FINALIZE

After a node reaches a terminal state, propagation repeatedly examines blocked nodes.

- If all dependencies succeeded, the node becomes available and emits `node_ready`.
- If any dependency is terminal and not succeeded, the node becomes skipped and emits `node_skipped`.
- Otherwise it remains blocked.

The repeated pass computes the least fixed point because marking one node skipped may make its descendants permanently blocked.

A run succeeds exactly when all nodes are terminal and every node succeeded. It fails when all nodes are terminal and any node failed, skipped, or was canceled due to failure. Explicit operator cancellation yields the distinct canceled run state.

9. Refinement and correctness argument

9.1 Abstraction relation

Let $((C))$ map an operational configuration to a denotational observation containing:

- the immutable run input and plan;
- successful node artifact values and digests;
- terminal failure or cancellation classification;
- semantic cache graph;
- current alias values;
- the causal event history, quotienting independent event order where required.

Operational metadata such as the current worker identity or exact lease expiration is omitted from the value observation, but it remains relevant to which transitions are legal.

The intended refinement claim is:

Every accepted Store transition preserves plan causality and corresponds either to an internal operational step with no artifact-denotation change, or to one admissible denotational computation step.

This report gives a paper proof sketch and executable tests, not a mechanized simulation proof.

9.2 Internal steps

CLAIM, HEARTBEAT, retry scheduling, and lease recovery are internal with respect to the successful artifact denotation. They change ownership, time, attempt history, and traces. They do not create a successful artifact value.

CACHE-HIT is denotationally equivalent to execution under Proposition 7.2. SUCCEED adds the node value. FAIL selects a failed branch of the computation relation. READY exposes an event already permitted by causality. PUBLISH changes the mutable alias observation under its conditional transition.

9.3 Linearization points

Each concurrent operation has a linearization point in the Store transaction.

- Run creation linearizes at the unique run/dedup insert.
- A claim linearizes at the conditional node update that increments the fence.
- Heartbeat linearizes at the fenced expiration update.
- Completion or failure linearizes at the fenced state transition.
- Cache insertion linearizes at the unique semantic-key row.
- Alias publication linearizes at the compare-and-swap update or atomic rename under lock.
- Event append linearizes with the per-run event counter increment.

These points allow a concurrent execution to be explained as a legal sequential history of transitions, even though handlers execute outside the database transaction.

9.4 Database interpreter obligation

The in-memory Store holds one mutex across each method and therefore realizes atomic transitions directly. A PostgreSQL interpreter must preserve the same preconditions with row locks and conditional updates. The SQL schema is not the semantics by itself; the combination of schema, transaction code, and constraints is.

A production implementation should treat the in-memory Store's observable behavior as a conformance target. Differential tests can run the same transition sequences against both stores and compare normalized snapshots and events.

10. Safety and liveness properties

10.1 Dependency safety

Theorem 10.1. A node can be claimed only after every direct dependency has succeeded.

Proof sketch. At creation, every non-root node is blocked. The only transition from blocked to available is propagation's all-dependencies-succeeded branch. No failure or recovery transition creates availability for a blocked node. CLAIM requires available state. Induction over transitions establishes the invariant.

This theorem is stronger than physical queue ordering. Even if a duplicate child dispatch exists in River, the control-plane start transaction must verify readiness before granting a fence.

10.2 Fence monotonicity and stale-worker exclusion

Lemma 10.2. For a fixed run/node pair, the fence value is strictly increasing across claims.

The claim transaction increments the persisted value and no other transition decreases it.

Theorem 10.3. After ownership transfers to a later claim, no completion or failure from an earlier claim can be accepted.

Proof sketch. Completion and failure require equality of run, node, worker, attempt, and fence with the current running record, plus a live lease. A later claim has a greater attempt and fence. Cancellation or terminal transition also violates the running-state predicate. Therefore the old capability cannot satisfy the precondition.

This protects the control database from a paused process that resumes after recovery. It does not by itself reverse an external side effect already performed by the stale worker.

10.3 Terminal monotonicity

Theorem 10.4. Once a run or node enters a terminal state, no legal transition returns it to a nonterminal state.

Proof sketch. All mutating Store methods check terminal/running state preconditions. Propagation operates only on blocked nodes. Retry operates only while processing a current running lease. Administrative rerun creates a new run or explicit new dispatch generation; it does not mutate terminal history into a fresh attempt.

Terminal monotonicity makes event replay and incident analysis stable.

10.4 Single accepted outcome per attempt

Theorem 10.5. At most one success or failure outcome is accepted for a particular (run,node,attempt,fence).

The first accepted outcome changes node state from running and clears the lease. Every later submission fails the current-lease predicate. An acknowledgment may be delivered zero, one, or several times, but the state transition occurs once.

10.5 Event-prefix integrity

Theorem 10.6. Event sequence numbers for one run form a gap-free prefix of the positive integers in the reference interpreter.

The in-memory append increments one run-local counter while holding the Store lock. The PostgreSQL helper increments event_count while locking/updating the run and inserts the event in the same transaction. A rolled-back transition rolls back both. Archival may move old events, but the authoritative logical sequence remains stable.

A global order across runs is neither required nor claimed.

10.6 Cache functionality

Theorem 10.7. At every reachable state, one semantic key maps to at most one output digest.

Proof sketch. The cache is a map in memory and a primary-key table in PostgreSQL. Completion checks an existing value under the same atomic transition. Equal digest is accepted; unequal digest returns `ErrResultConflict` and does not commit success. Concurrent insertions are serialized by the map lock or database uniqueness/row lock.

A conflict is evidence of an incomplete key, an unpinned dependency, nondeterminism, or corruption. It should page or quarantine the node kind rather than be treated as an ordinary transient.

10.7 Conservative cost-budget safety

Let (C_c) be consumed measured cost and (C_r) reserved declared maximum cost.

Theorem 10.8. If every handler's actual cost is no greater than its declared maximum, then the Store never starts work whose worst-case completion would make (C_c+C_r) exceed a positive run maximum.

Proof sketch. Claim admission checks

$[C_c+C_r+g_{CC}_{\{}}]$

before adding (g_C) to reservation. Completion or failure removes the reservation and adds actual cost (cg_C) . Concurrent claims are serialized by the Store transition. Induction preserves the bound.

The Store rejects a reported cost above grade. That detects protocol violation after execution; it cannot prevent already incurred external spend. Grades should therefore be derived conservatively from batch size and provider pricing.

10.8 Concurrency-key exclusion

Theorem 10.9. At most one running node with a given non-empty concurrency key is accepted by a conforming Store.

The in-memory Store checks and updates under one lock. PostgreSQL uses a partial unique index as a second line of defense and should acquire a transaction-scoped advisory lock before the conditional update. The property is global across runs.

Publication keys should normally be scoped as `publish:<system>:<corpus>:<environment>`.

10.9 Alias linearizability and revision monotonicity

Theorem 10.10. Under an atomic alias lock/update, successful publications to one alias are linearizable by generation.

Each success reads one current generation and writes exactly the next generation while excluding concurrent writers. A compare-and-swap conflict has no write effect.

Theorem 10.11. When nonzero ordered revisions are required, an accepted alias revision never decreases, and an equal revision can succeed only as an idempotent replay of the same digest.

This follows directly from the publication precondition. The property closes a subtle outbox race: revision 11 arriving after revision 12 cannot roll the active alias backward even if it creates a later run.

10.10 Trigger deduplication and contiguous adoption

Theorem 10.12. Reordering or replaying the same normalized change set produces the same batch deduplication key.

Changes are sorted by stream, revision, and event identifier; duplicate event identifiers are removed; entity and event sets are sorted; the key hashes the normalized stream, revision interval, and event identifiers.

Theorem 10.13. `CoalesceContiguous` emits no revision greater than the first missing revision after the durable committed cursor.

The algorithm expects `committed+1`, groups all events sharing that revision, increments the expectation only after observing it, and stops when the next available revision is greater. Later events remain unprocessed. The caller advances the durable coalescer cursor only after run adoption commits.

10.11 Run finalization safety

Theorem 10.14. A run is marked succeeded only when every node succeeded.

Finalization returns while any node is nonterminal. It marks failure if any terminal node is failed, skipped, or canceled. The remaining case is that every node succeeded.

10.12 Liveness

Safety does not imply progress. A liveness result requires environmental assumptions.

Assume:

1. the control database eventually becomes available;
2. at least one compatible worker for every required queue polls fairly;
3. database time advances and lease recovery runs fairly;
4. each handler attempt terminates, crashes, or loses its lease in finite time;
5. retry policies and run budgets are finite;
6. concurrency-key holders eventually complete or expire;
7. no operator leaves the relevant queue permanently paused;
8. every available physical dispatch can eventually be observed, either by polling or reconciliation.

Theorem 10.15 (Eventual terminality under fairness). Under these assumptions, every finite run eventually reaches succeeded, failed, or canceled.

Proof sketch. The DAG is finite. Every root is eventually claimed or terminally rejected by budget. Each running attempt eventually completes, fails, or expires. A retry can occur only finitely many times because attempts and budgets are finite. Thus each node eventually reaches a terminal state or makes a child available. Induction along a topological order yields terminality for all nodes and then the run.

The theorem does not guarantee success. Permanent defects, quality rejection, exhausted budgets, and operator cancellation are valid terminal outcomes.

10.13 Exactly-once boundary

No theorem in this report claims exactly-once execution of arbitrary handlers. A worker may call an external service, lose the response, expire, and retry. The provider may have committed the first request.

The system instead offers patterns for observable idempotence:

- pure or content-addressed computation;
- provider-supported idempotency keys derived from semantic identity;
- transactional outbox/inbox when both effects share a database;
- compare-and-swap with expected generation or digest;
- fence-aware conditional mutation;
- reconciliation by querying provider operation identity;

- immutable result adoption after digest verification.

When none applies, an ambiguous outcome must become a distinct reconciliation state or a permanent operator-visible failure. Blind retry of an unknown non-idempotent effect is unsound. This follows established transaction-processing experience: distributed atomicity cannot be wished into existence across autonomous services ([Garcia-Molina and Salem 1987](#); [Helland 2007](#)).

11. Change capture and source consistency

11.1 The dual-write problem

A database mutation that changes retrieval content and a scheduler submission are two effects. When the source database and control database are different systems, no ordinary application transaction can atomically commit both. Writing the domain row and then enqueueing can lose the job after the first commit. Enqueueing first can build an index for a mutation that later rolls back. Retrying either sequence can create duplicates.

GEC uses MySQL for application data while the proposed control plane and River profile use PostgreSQL. The design therefore uses a transactional outbox in MySQL. The domain transaction updates business rows, allocates a stream revision, and inserts one or more outbox events before committing. A relay copies committed events to a deduplicating PostgreSQL inbox at least once.

TTC may use an immutable repository commit as its source cursor, in which case a repository webhook or polling reconciler can submit (commit SHA, configuration digest). If TTC also indexes mutable application tables, it should use the same source-side outbox pattern in its database.

11.2 Source revision

Each independently ordered source stream has a key

```
(source, system, corpus)
```

and a monotonically increasing positive revision. One transaction may produce several events at the same revision when it changes several relevant entities. The revision is allocated inside the same database transaction as the domain mutation and outbox rows; rollback therefore rolls back the revision increment.

A revision is not merely a timestamp. It provides a total order within the stream, detects relay gaps, supports contiguous adoption, and supplies a monotone publication guard. Wall-clock timestamps remain useful for latency measurement but cannot safely order concurrent transactions across machines.

The supplied MySQL migration uses a `rag_index_revision` row and InnoDB transaction. The application allocates the next value, writes the domain mutation and `rag_index_outbox` rows, and commits. A PostgreSQL source can use an identity/sequence or an application stream row under transaction.

11.3 Relay protocol

The relay performs the following loop.

1. Read a bounded set of unrelayed outbox rows in source order.
2. Insert normalized records into `ragjobs_trigger_inbox` using unique `(source,event_id)` conflict suppression.
3. Commit the target transaction.
4. Mark source rows relayed, or leave them pending if acknowledgment is ambiguous.
5. Repeat.

The relay is deliberately at least once. If the target commit succeeds but the relay loses the acknowledgment, the next insert is a no-op under the unique constraint. Source rows should not be deleted merely to reduce backlog; retention or archiving follows confirmed relay and a safety window.

A relay should expose source high-watermark, target high-watermark, oldest pending age, copy rate, duplicate rate, and errors. The critical service-level objective is not raw queue depth but the age and revision lag of the oldest unadopted index-relevant change.

11.4 Contiguous adoption

An inbox may contain revision 10 and 12 while revision 11 is delayed by relay batching or a transient error. A coalescer that simply takes minimum and maximum would falsely claim a batch covering 10-12 and could mark revision 12 processed before 11 arrives.

The production coalescer keeps a durable committed cursor per stream. It asks for the contiguous prefix beginning at committed+1. All events sharing a revision are included. At the first gap it stops, leaving later records unprocessed. It creates one deterministic batch and one run under a unique deduplication key. Only after the run has been durably adopted does it advance the coalescer cursor.

The reference package contains both `Coalesce`, which deterministically splits arbitrary input at gaps, and `CoalesceContiguous`, which enforces the durable prefix rule. The latter is the production primitive.

A batch identity includes stream, from/to revision, and normalized event identifiers. Replaying the same inbox records yields the same key. Payload order and relay order do not change it.

GEC Transactional Outbox and Index Publication

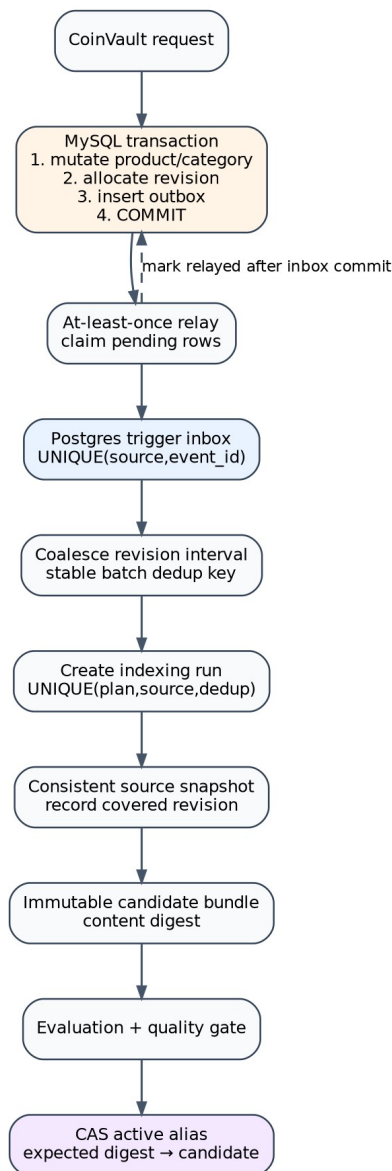


Figure 5. Transactional outbox, at-least-once relay, contiguous coalescing, snapshot build, and monotone alias publication.

11.5 Snapshot protocol for mutable databases

Receiving a committed outbox revision means its source transaction is visible to a new database snapshot. The snapshot handler should:

1. begin a consistent read transaction;
2. read the source stream's current revision inside that snapshot;
3. ensure it is at least the requested contiguous batch revision;
4. read all index-relevant rows and versioned curated documents under the same snapshot;
5. normalize and write a content-addressed snapshot artifact;
6. record the actual snapshot revision, table/query fingerprints, source schema version, and configuration references;
7. commit or end the read transaction.

The actual snapshot may include revisions later than the triggering batch because new domain commits can occur before the consistent read begins. This is acceptable if the snapshot artifact records the actual revision and publication uses it. It is not acceptable to cache the snapshot only under the older requested cursor, because rerunning that cursor later could observe different state. The generic plan therefore disables semantic caching for mutable snapshots. Downstream nodes key on the actual snapshot digest.

A stricter system may use temporal tables, binlog positions, or database time travel to materialize exactly revision (r). In that case the snapshot can be declared immutable and cacheable.

11.6 Repository snapshots

For a source repository, the cursor is an immutable commit object identifier plus any submodule or dependency lock information. The snapshot handler verifies the commit exists, checks out or reads it without mutable working-tree state, applies a versioned inclusion policy, and writes a source manifest containing file digests.

A repository snapshot is cacheable when the handler version, commit, inclusion policy, and relevant toolchain inputs are part of semantic identity. Branch names are not immutable cursors and should be resolved to a commit before run creation.

11.7 Configuration changes and backfills

Not all invalidation originates in source rows. Changes to chunking, analyzer configuration, embedding model, prompt, representation schema, evaluation set, or quality policy require explicit versioned triggers.

The run's configuration hash should be a digest over all value-affecting settings, including transitive files. A configuration-only backfill uses a new deduplication key and configuration digest even if the source revision is unchanged. Old bundles remain reachable until the candidate is evaluated and promoted.

A schema migration that changes source interpretation should create a new handler version or configuration epoch. Reusing a semantic key across incompatible schemas is prohibited.

11.8 CDC as an alternative relay

Log-based change data capture can replace an application-managed relay when operated reliably. It does not remove the need for normalized stream order, deduplication, contiguous adoption, and run identity. CDC offsets become part of the source protocol, and snapshots must reconcile the initial snapshot boundary with the log position.

The transactional outbox is preferred initially because the relevant event is application-semantic: a product or curated document changed in a way that affects a named corpus. Raw row CDC can generate excessive, poorly scoped rebuilds unless a normalization layer reconstructs that meaning.

12. Retry, leases, fencing, cancellation, and budgets

12.1 Retry layers

RAG handlers often contain an inner retry layer. An embedding batch may retry individual provider requests; a database reader may retry one transaction; an object-store client may retry an idempotent upload. The outer scheduler retries the node after process death, exhausted inner retry, database outage, or a classified whole-node failure.

Nested retry is valid only when the layers have distinct scopes and bounded multiplication. A practical rule is:

- inner retry handles short-lived request-level transients with small limits;
- outer retry handles process and node-level recovery with larger delays;
- provider rate-limit hints flow outward when inner retry stops;
- the product of worst-case attempts is included in cost and latency planning.

Returning an error to River while separately creating a delayed ragjobs attempt violates this rule. Exactly one system owns the physical retry clock.

12.2 Deterministic backoff

Deterministic jitter serves two goals. It spreads many failed runs so they do not synchronize on the same provider, and it makes tests and incident reconstruction repeatable. The seed uses run, node, and attempt identity, so separate runs receive different schedules while a replay of one run computes the same delay.

Backoff is capped. Rate-limit responses with an explicit retry-after delay override the generic schedule. Conflict errors may use a shorter delay, but persistent semantic conflicts should become permanent rather than spin.

The next availability time is durable. A worker restart does not reset backoff.

12.3 Lease duration

Lease duration should exceed normal heartbeat and transient scheduler pauses but remain short enough for acceptable recovery. The worker heartbeats at approximately one third of the lease duration. A starting value for long indexing tasks is two minutes with heartbeats every forty seconds, adjusted after observing runtime pauses and database latency.

A lease is not the handler timeout. The timeout bounds one attempt's cooperative execution. The lease bounds authority and is extended while work remains healthy. A handler can have a four-hour timeout and a two-minute renewable lease.

Long non-interruptible external calls require careful lease selection or a provider operation protocol. If a single call can block longer than the lease and cannot be canceled, the worker may lose authority while the provider continues. The external request should then use a stable idempotency key and be reconciled on retry.

12.4 Fencing beyond the control database

The Store fence prevents stale control transitions. External mutable resources need their own fence checks.

For alias publication, the handler uses the lease fence as provenance and performs compare-and-swap on expected alias state plus monotone revision. For an application database mutation, a target row can store the latest accepted fence or generation and reject a lower value. For a provider supporting idempotency keys, the semantic key or `(run,node,dispatch-generation)` can name the operation.

A filesystem lock on one host is sufficient only for a single shared filesystem with correct locking semantics. Multi-region object storage should use a database alias row or native conditional object update.

12.5 Ambiguous completion

Suppose a worker writes an immutable artifact and calls Store completion. The database commits, but the response is lost. The worker retries or its lease expires. The old completion cannot be accepted again, but no data is lost: the node is already succeeded.

Suppose alias publication commits but Store completion is lost. A later attempt presents the same digest and source revision. The alias operation recognizes the exact pair as an idempotent replay and returns the existing generation. The worker can then complete the new attempt. If a newer revision is active, the older attempt receives a stale-revision result and must reconcile rather than overwrite.

This distinction between effect idempotence and attempt identity is essential. A new attempt has a new fence but may lawfully adopt the same already-committed semantic effect.

12.6 Cancellation semantics

Cancellation has three layers.

The durable control transition prevents any further accepted node mutation. The transport cancellation tries to stop queued or running physical jobs. The handler context asks cooperative code and network clients to stop.

The first layer is authoritative. The latter two reduce wasted work but may lag. Cleanup of temporary artifacts should be idempotent and may be performed by a separate reconciler after cancellation.

Operator cancellation records actor, reason, and time. Automated cancellation records its policy source, such as deployment drain, deadline, or superseded revision. Cancellation should not delete evidence.

12.7 Supersession

A high-change corpus may create a new run while an older build is active. Three policies are useful.

Queue all revisions. Appropriate when each index state is independently valuable for audit or when changes are sparse.

Coalesce before run creation. Preferred default. A debounce window gathers a contiguous prefix and builds the latest snapshot.

Supersede active candidates. A newer run can request cancellation of an older non-published run for the same corpus. This saves cost but must not cancel a run whose artifacts are needed for comparison. Publication monotonicity remains the final guard even when cancellation races.

The reference plan exposes a publication concurrency key but leaves supersession policy to the control service because it depends on business latency and evaluation requirements.

12.8 Budgets

A run budget contains maximum total attempts, maximum cost microunits, and an optional deadline. Nodes declare a maximum cost. Claim reserves that maximum; completion replaces the reservation with actual reported cost.

Cost microunits are an internal accounting unit. An organization can define one unit as a micro-dollar, a provider token-normalized unit, or a composite charge. What matters is stable versioned conversion and conservative bounds.

Budget exhaustion is a domain-visible terminal class, not an infrastructure retry. An operator may create a new run with a larger budget after reviewing evidence. Mutating the historical run budget would make its decision trail harder to interpret.

12.9 Terminal work and dead letters

A terminal node is retained in the control plane with all attempts and evidence. A separate dead-letter queue is optional; the run itself is the richer dead-letter record. River Pro or another transport may provide a physical dead-letter feature, but application remediation should be driven by the `ragjobs` terminal state and error class.

Administrative retry should generally create a new run or an explicit new dispatch generation with an audit event. Reopening a terminal node in place would violate terminal monotonicity and complicate event semantics. A future first-class `retry-from-node` operation can derive a new run that reuses successful upstream semantic results.

13. Artifact, cache, and publication architecture

13.1 Content-addressed objects

A content-addressed object reference is

```
(digest, size, media-type, storage-key)
```

The file reference implementation streams bytes into a temporary file while computing SHA-256, synchronizes the file, and renames it into a path derived from the digest. If the object already exists, it verifies the existing bytes. The rename and parent-directory synchronization provide local-filesystem durability under the documented filesystem assumptions.

Object storage uses the same logical protocol with multipart upload, checksum verification, and an immutable key. The storage key is an implementation detail; the digest is semantic identity.

PutBytes uses a binary-safe byte reader. Verify recomputes digest and optional size. Path and alias validation reject traversal.

13.2 Artifact envelopes

Node output JSON should normally be an envelope rather than a large inline body. A recommended envelope is:

```
{
  "schema": "rag/chunk-set/v2",
  "digest": "sha256:...",
  "size": 123456,
  "media_type": "application/vnd.example.rag-chunks+json",
  "storage_key": "objects/sha256/ab/ab...",
  "metadata": {
    "source_revision": "1042",
    "configuration_digest": "sha256:..."
  }
}
```

The envelope itself is canonical JSON and receives a node output digest. The referenced object also has a digest. This two-level form permits a stable typed descriptor to carry metadata while preserving byte integrity.

13.3 Bundle manifest

An index bundle manifest should include:

- bundle schema and builder version;
- source snapshot digest and actual revision;
- corpus and chunk-set digests;
- lexical index digest and analyzer configuration;
- vector index digest, embedding model revision, dimension, and metric;
- representation and normalization versions;
- build plan and run identifiers;
- creation time as provenance, not identity;
- verification results;
- optional evaluation evidence references.

ragkit/rag/indexbundle already implements content-derived bundle identity and atomic directory publication. The scheduler should reuse that contract and avoid introducing a second mutable bundle format.

13.4 Semantic cache index

The semantic cache maps an invocation key to an output envelope and digest. It is a control-plane index over immutable objects, not a mutable blob cache. Deleting a cache row prevents reuse but does not necessarily delete its artifact. Artifact garbage collection uses reachability and retention, not cache eviction alone.

Cache provenance records source run and node. A cache hit emits a distinct event. Operators can therefore answer whether a stage executed or reused prior work.

A cache invalidation action should identify key, actor, reason, and time. Invalidating by handler kind/version or configuration epoch can be implemented as a query that marks matching entries ineligible without destroying evidence.

13.5 Publication aliases

An alias record contains name, artifact digest, generation, update time, run, plan, node, fence, cursor, optional numeric revision, and metadata. The file implementation stores JSON under an alias path and updates it under an exclusive lock using temporary-write, synchronization, and rename. PostgreSQL stores the same logical value in one row.

Readers follow a two-step protocol:

1. resolve the alias once and obtain digest/generation;
2. open and verify the immutable bundle by digest.

A request should not repeatedly resolve the alias mid-query. A serving process may keep the previous bundle open while loading and smoke-testing the new one. Only after the new bundle opens successfully should it retire the old handle.

13.6 Publication quality gate

The publication handler requires evidence that verification passed and, when configured, that the named quality policy accepted the candidate. It should not recompute the whole evaluation. It validates evidence digests, candidate digest, policy version, and source revision, then executes alias compare-and-swap.

The expected current digest or generation should be captured when the publication attempt establishes its precondition. A conflict means another publication won. The handler then determines whether the active alias is already the desired digest, is a newer revision, or represents an unrelated concurrent change. The first is idempotent success; the second is stale rejection; the third may be retryable conflict after policy review.

13.7 Rollback

Rollback does not mutate an immutable bundle. It creates an audited publication operation that advances the alias to a prior known-good digest with a new generation. For an ordered source stream, operational rollback may intentionally move to an older content revision, so it must use a privileged override protocol distinct from ordinary monotone automatic publication. The override records incident, actor, reason, prior generation, and target digest.

After rollback, automated publishers should not blindly resume. The stream may need a quarantine or minimum accepted revision policy until the defect is corrected.

13.8 Garbage collection

Garbage collection is mark-and-sweep with a safety delay.

Roots include active aliases, rollback-retention aliases, nonterminal runs, retained run outputs, cache rows, evaluation evidence, manually pinned artifacts, and incident quarantines. The collector marks transitively referenced objects, records an unreachable candidate time, waits a safety interval, rechecks reachability, and then deletes.

Published artifacts are never collected merely because they are old. Failed candidate artifacts often have higher diagnostic value than successful intermediate objects and may deserve longer retention.

13.9 Corruption and quarantine

Digest verification detects accidental corruption. A mismatch should quarantine the object key, fail verification permanently, invalidate cache entries that reference it, and prevent publication. If an active bundle becomes unreadable, serving should fall back to the previous verified generation where possible and open an incident.

Digest algorithms are versioned by prefix. SHA-256 is adequate for the current design. A future migration can support multiple algorithms without changing the logical artifact API.

14. Production indexing workflow

14.1 Stage contracts

The reference indexing plan uses the following contracts.

Node	Input type	Output type	Default cache
snapshot	source cursor and run values	rag/snapshot-ref	off for mutable source; on for immutable source
extract-corpus	snapshot	rag/corpus	content
chunk-corpus	corpus	rag/chunk-set	content
build-lexical	chunk set	rag/lexical-index	content
embed-dense	chunk set	rag/vector-index	content under pinned model/config
assemble-bundle	lexical and vector indexes	rag/index-bundle	content
verify-bundle	bundle	rag/verified-bundle	content
evaluate-candidate	verified bundle	rag/evaluation-report	off unless deterministic
quality-gate	evaluation report	rag/gate-decision	content
publish	terminal candidate/decision	rag/publication	off
cleanup	prior publication dependency	no semantic output	off

The plan constructor statically verifies direct port-type coverage and rejects a quality gate without evaluation.

14.2 Snapshot

The snapshot node is the only stage permitted to read mutable source truth directly. All later stages consume immutable artifacts. This creates a clean consistency boundary.

For GEC, the snapshot includes normalized MySQL row data and curated SQL-document library version under a consistent read. For TTC repository indexing, it includes a commit-addressed file manifest. Sensitive source fields that should never enter retrieval are removed or transformed here, before artifacts become broadly reusable.

The snapshot output records source schema and extraction-policy versions. A source row count alone is not a fingerprint.

14.3 Corpus extraction

Extraction converts source-specific values into a shared canonical document model: stable document identifier, title, body or sections, source URI, domain metadata, access-control labels if relevant, and content digest.

Stable identifiers must survive harmless reordering and retries. Deletions are represented by absence from the immutable snapshot, not by mutating a previous corpus artifact. The extractor sorts documents and canonicalizes text according to a versioned policy.

A document-level inner cache may accelerate extraction, but the node output remains one corpus artifact whose digest commits to the whole ordered set and manifest.

14.4 Chunking and representations

Chunking is a semantic transformation, not a formatting detail. The configuration identifies algorithm, heading behavior, tokenization, overlap, maximum length, metadata projection, and representation generation. Changing any of these changes the configuration digest or handler version.

Each chunk receives a stable identifier derived from document identity, content/section fingerprint, and chunker version. Stable chunk IDs improve evaluation comparison and future segment-level reuse.

The chunk-set manifest records ordering, representation schemas, and per-chunk digests. It is the common input to lexical and dense branches.

14.5 Lexical branch

The lexical branch builds analyzer-specific postings and supporting metadata from the chunk set. It should write to an isolated temporary or content-addressed location and produce a descriptor only after the index opens successfully.

Analyzer rules, stop words, stemming, field boosts, language, and library version participate in semantic identity. A library upgrade that changes index bytes or query behavior requires a handler or configuration epoch change.

14.6 Dense branch

The dense branch may be the most expensive stage. It declares external embedding calls and a conservative cost maximum. It can use ragkit/flow or an equivalent inner batch engine for per-item content caching, request retry, batching, and repair.

The outer handler should group inputs deterministically and produce a vector manifest keyed by chunk digest. Exact embedding model revision, dimension, normalization, provider, endpoint class, and representation text must be pinned. A provider's moving model alias is not a stable semantic dependency.

When a provider accepts idempotency keys, use a key derived from model revision and chunk/representation digest. On ambiguous timeout, query or replay under the same key.

14.7 Assemble and verify

Assembly combines lexical and vector descriptors with the corpus/chunk manifest into a ragkit-compatible immutable bundle. It must verify that both branches cover the same chunk-set digest and expected item set. Mismatched source fingerprints are a permanent semantic error.

Verification is a separate node so a bundle produced by a recovered or reused assembly still undergoes a named check. Verification should:

- validate the manifest and every referenced digest;
- open lexical and vector indexes;
- check dimensions, counts, and schema versions;
- run deterministic smoke queries;
- confirm no temporary paths are referenced;
- emit a verified-bundle descriptor with evidence.

Verification is cacheable because it is a deterministic function of immutable bundle bytes and verifier version.

14.8 Evaluate and gate

Evaluation may be embedded in the indexing run for required pre-publication evidence or submitted as a separate plan over the candidate digest. The separate-plan option is useful for expensive suites, multiple experimental arms,

or manual candidate comparison. The indexing run then waits for or references the resulting signed evidence through a gate node.

The gate policy should include hard floors, regression tolerances, sample-size requirements, and route-specific rules. A policy can reject a candidate even when average metrics improve, for example if a critical lexical route falls below a floor.

14.9 Publish and cleanup

Publication is queued separately from CPU indexing, embedding, and evaluation. Its concurrency key serializes one corpus/environment. The handler verifies candidate and gate evidence, checks source revision, and advances the alias.

Cleanup runs after publication and may delete temporary workspaces, expired provider staging files, and uncommitted scratch data. It must never delete immutable objects merely because the current run did not publish; retention and garbage collection decide those objects later.

A cleanup failure should generally be `ContinueRun` or handled by a separate janitor in production, because failing the already-published run can confuse operators. The reference plan currently uses `fail-run` uniformly for simplicity; an application adapter should choose a cleanup policy explicitly.

14.10 Incremental-segment evolution

The immutable rebuild is an architectural baseline, not a permanent performance ceiling. A later segment design can preserve the same plan semantics:

```
snapshot delta
-> normalize changed documents
-> build immutable lexical/vector segments
-> assemble manifest referencing old + new segments and tombstones
-> compact as a separate background plan
-> verify/evaluate/publish manifest alias
```

The published object remains an immutable manifest. Compaction creates a new candidate rather than mutating active segments. Semantic keys use segment input digests. This retains rollback and schedule-independence arguments while reducing rebuild cost.

15. Evaluation batch semantics

15.1 Evaluation as a first-class workflow

Evaluation is not a post-hoc script. It consumes named immutable candidate and baseline bundles, a versioned case set, route definitions, metric implementations, optional judge configuration, and a sharding policy. Its outputs are immutable evidence used by quality gates and experiments.

A dedicated evaluation plan is:

```
enumerate-cases
  -> evaluate-0000 ... evaluate-N
  -> aggregate
  -> [compare]
```

The plan constructor supports up to 1024 shards and uses stable zero-padded shard identifiers.

15.2 Deterministic partitioning

`enumerate-cases` loads a versioned suite and emits a partition manifest. Case assignment must be deterministic, for example

[(q)=H((q))N.]

The partition artifact records suite digest, case identifiers, shard count, and assignment. Changing shard count changes the enumeration output and shard semantic inputs, while aggregate semantics remain comparable.

Each shard consumes the same partition manifest and its explicit shard number. It writes per-case evidence and summary statistics. Shards are independent and can be retried or reused when deterministic.

15.3 Metrics

Retrieval evaluation can include recall at (k), reciprocal rank, nDCG, exact source match, route selection, latency, and domain-specific coverage. Generation evaluation may include deterministic rule checks and externally judged criteria.

Every metric result should carry implementation version and denominator. Aggregate reports must distinguish missing, skipped, invalid, and failed cases rather than silently dropping them. Confidence intervals or bootstrap distributions are appropriate when policy depends on small differences.

Averages alone are inadequate for critical corpora. Reports should include per-slice metrics for route, category, language, source type, difficulty, and protected operational cohorts where applicable.

15.4 Stochastic judges

An LLM judge introduces nondeterminism, provider drift, and cost. Default shard cache is therefore off. To make a judged run reproducible enough for governance, record:

- provider and exact model revision where available;
- system and rubric prompt digests;
- decoding settings and seed if supported;
- input case and candidate response digests;
- raw judge response and parsed decision;
- retries and provider request identity;
- judge cost and latency;
- evaluator code version.

Organizations may choose a recorded-experiment semantics in which the first accepted judge output is immutable evidence and can be reused by explicit policy. That is not the same as asserting the judge is a mathematical function. The implementation's `CacheShards` and `CacheEvaluation` flags make this choice explicit.

15.5 Aggregation

Aggregation is deterministic over immutable shard reports. It verifies complete expected shard coverage, rejects duplicate or conflicting case evidence, combines metrics using declared weighting, and emits one report digest.

Because aggregate input digests identify every shard, aggregation is safely cacheable even when shards are not. A failed shard prevents aggregate readiness under the default fail-run mode. A diagnostic plan may use continue-run and an aggregator that represents missing shards explicitly, but that is a distinct contract.

15.6 Baseline comparison

Comparison consumes candidate aggregate evidence and a named baseline bundle/report. It computes absolute and relative deltas, slice regressions, and policy features. The baseline identity must be immutable; `production` as an alias is resolved to a digest before run creation.

Comparisons should avoid repeated significance fishing. The quality policy defines the metrics and tolerances before the run. For LLM judge outcomes, paired evaluation on the same cases reduces variance.

15.7 Quality policy

A gate policy can combine:

- hard structural requirements;
- minimum recall or nDCG floors;
- no-regression tolerances against baseline;
- critical-case zero-failure rules;
- maximum latency and cost;
- minimum valid sample counts;
- manual approval requirements for specified change classes.

The policy returns a decision artifact with machine-readable reasons. Automatic publication reads this artifact. A dashboard may render it, but the dashboard is not the policy engine.

15.8 Experiment linkage

TTC already records experiments. The scheduler run should be the durable orchestration parent, while experiment records contain RAG-specific arms, metrics, and artifacts. Both identifiers cross-link. A single scheduler evaluation run may create several experiment arms; an experiment should not be the only record of leases, attempts, or source change adoption.

16. PostgreSQL control-plane interpreter

16.1 Data model

The supplied migrations define runs, nodes, attempts, events, semantic cache, artifact aliases, trigger inbox, optional River dispatch links, and worker heartbeats.

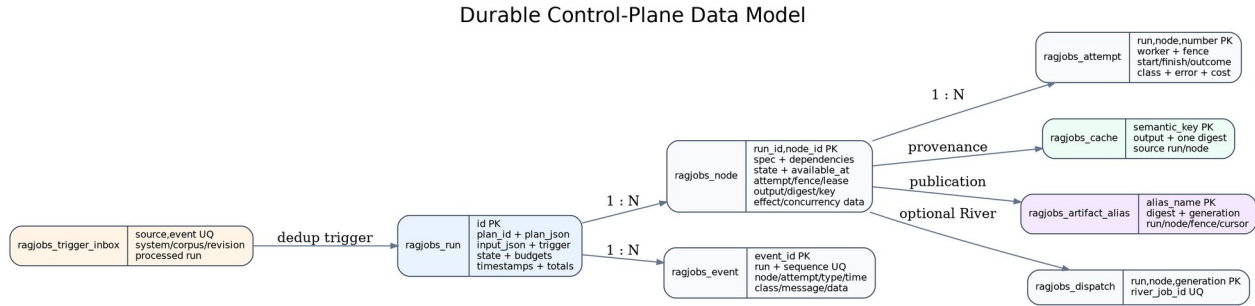


Figure 6. PostgreSQL control-plane data model. Large artifacts remain outside the relational control plane and are referenced by immutable digests.

ragjobs_run stores canonical plan and input JSON, source trigger fields, state, budgets, timestamps, event count, attempts, and cost. **ragjobs_node** stores the normalized node specification, dependency array, readiness, lease/fence, output descriptor, semantic key, declared grade, and failure policy. Attempt rows preserve every owner/fence outcome. Event rows form a per-run ordered audit stream.

The semantic cache has one row per key. The alias table stores generation and source revision. The trigger inbox deduplicates relay events. The optional dispatch table links a semantic node dispatch generation to a River job identifier without making River’s row the semantic source of truth.

16.2 Run creation transaction

Run creation finalizes the plan in application code, begins a transaction, attempts the deduplicated run insert, inserts all node rows, appends creation and root-ready events, and commits. In River mode it also inserts root River jobs through River’s transaction-aware client before commit.

A duplicate trigger returns the existing run. The caller should not enqueue another physical root job unless reconciliation proves the existing run lacks its required dispatch generation.

16.3 Claim transaction

The supplied `ragjobs_claim_nodes` helper demonstrates the hot claim transaction:

1. select candidate available nodes ordered by priority and age;
2. lock rows with `FOR UPDATE SKIP LOCKED`;
3. lock and recheck the owning run;
4. enforce deadline, attempt, and cost admission;
5. acquire an advisory lock and recheck any concurrency key;
6. increment attempt and fence;
7. reserve declared cost;
8. insert attempt and event;
9. return the lease fields.

PostgreSQL documents `SKIP LOCKED` as suitable for avoiding lock contention with multiple consumers of a queue-like table, while warning that it gives an inconsistent view for general-purpose queries ([PostgreSQL Global](#)

[Development Group 2026](#)). This is exactly the intended use: candidate selection can skip locked work, and each selected row is rechecked under transaction.

The helper is a contract example, not a claim that all Store methods have been live integration-tested in this environment. Completion, failure, recovery, and cancellation must implement the documented state machine transactionally.

16.4 Completion transaction

A completion transaction locks run and node, validates owner/attempt/fence/live lease, canonicalizes and verifies output, enforces cost grade, and serializes the semantic cache key. It finishes the attempt, releases reservation, records success, appends an event, activates children, finalizes the run if applicable, and commits.

In River mode, newly ready child River jobs and transactional job completion belong in this same database transaction. River documents transaction-aware enqueueing and transactional completion mechanisms ([River Queue 2026d](#), [2026e](#)). The adapter must use the exact API appropriate to its installed River version.

16.5 Failure and recovery transactions

Failure follows the same fenced precondition. It persists actual cost, retry time or terminal state, failure propagation, events, and finalization atomically.

The recovery process selects expired running rows with SKIP LOCKED, then invokes equivalent failure logic with lease-expired classification. It records an explicit recovery event and never silently resets a row.

Budget-deadline enforcement needs both claim-time checks and a periodic sweep. A run with no available nodes but a future retry can cross its deadline before the next claim. The sweeper finalizes it and fences any live work.

16.6 Isolation and constraints

READ COMMITTED is sufficient when every mutable row is explicitly locked, updates contain fenced predicates, cache and dedup identities have unique constraints, and the running concurrency-key partial unique index remains enabled. Using serializable isolation for all worker traffic would create avoidable aborts without replacing explicit protocol checks.

Constraints are defense in depth. They reject impossible states such as a running node without an owner/expiration or a succeeded node without a digest. Application transactions still carry the full transition semantics.

16.7 Notifications

LISTEN/NOTIFY can wake idle workers after run creation, retry availability, or recovery. It is never the durable queue. Notifications can be missed across disconnects and carry limited payload. Workers poll durable state on a bounded interval even when notifications are enabled.

16.8 Time

Production transactions should use PostgreSQL time consistently. Lease expiration comparisons based on independent worker clocks can fail under skew. The Go Store interface accepts time to make the executable specification deterministic; a PostgreSQL adapter may ignore caller time for authority and return database-generated timestamps in leases/events.

16.9 Partitioning and retention

At moderate scale, indexes on available nodes, expired leases, run state, event sequence, and source inbox are sufficient. High event volume can use time- or hash-partitioned event and attempt tables. Partitioning should not change per-run sequence semantics.

A starting retention policy is 30-90 days online for detailed events and attempts, longer for failed runs, and archival to immutable object storage. Run/node summaries and artifact lineage remain longer. Retention deletion is audited and never removes active alias provenance.

17. River versus a native queue

17.1 River capabilities relevant to the design

River is a PostgreSQL-backed Go job system with transaction-safe insertion ([River Queue 2026c](#), [2026d](#)). Its reliability documentation describes at-least-once behavior and retries when worker execution or completion persistence fails ([River Queue 2026g](#)). Unique jobs suppress duplicate insertions under selected dimensions but do not change at-least-once execution semantics ([River Queue 2026f](#)). River also supports cancellation and snoozing in its worker protocol, and documents transactional job completion ([River Queue 2026e](#)).

Core River is enough to transport ragjobs node dispatches. River Pro workflows can express DAG and fan-out/fan-in behavior, but they are not required because the open plan algebra remains the semantic source of truth ([River Queue 2026b](#)). Core periodic schedules are held in process memory; obligations that must survive scheduler restart should use a source outbox, an external durable scheduler, or a durable periodic feature ([River Queue 2026a](#)).

17.2 Direct Store profile

In the direct profile, workers poll ragjobs_node through the PostgreSQL Store. ragjobs owns claim, lease, retry timing, attempt history, and recovery. The benefits are one state machine, full control, and no queue-schema dependency. The costs are implementing and operating queue mechanics, maintenance, dashboards, and migration compatibility.

This profile is attractive when the organization wants one small specialized queue, has strong PostgreSQL expertise, and needs exact control over admission and resource scheduling.

17.3 River profile

In the River profile, ready semantic nodes create small physical dispatch jobs:

```
type NodeDispatchArgs struct {
    RunID      string `json:"run_id" river:"unique"`
    NodeID     string `json:"node_id" river:"unique"`
    DispatchGeneration uint64 `json:"dispatch_generation" river:"unique"`
}
```

The worker starts a control-plane attempt transaction, verifies the dispatch generation and readiness, obtains a semantic fence, executes the handler, and maps the result back to River behavior.

- transient or conflict: return an error for River retry;
- explicit non-attempt delay: snooze when policy calls for it;
- permanent, budget, or semantic conflict: persist terminal control state and cancel the physical job;
- operator cancellation: cancel both control run and River job;
- panic: let River record physical failure and persist a sanitized control event through middleware or recovery.

The control plane mirrors River attempt observations as needed, but River owns retry timing. It must not also set the semantic node available for an independent delayed claim.

17.4 Transaction boundaries in River mode

Run creation inserts control rows and root River jobs in one PostgreSQL transaction. Node success updates semantic state, inserts child dispatch records and River jobs, and completes the current physical job transactionally where the River API permits.

A reconciliation loop checks for ready semantic nodes lacking a live dispatch generation and for River jobs whose control node is already terminal. This handles operator mistakes and rare partial integration defects without redefining normal semantics.

Unique jobs are duplicate suppression, not semantic identity. The authoritative tuple is (run,node,dispatch-generation) plus the current fenced attempt. An operator-created redispatch increments generation.

17.5 Feature comparison

Concern	Direct PostgreSQL Store	River transport
physical claim/retry	implemented by ragjobs SQL	River
semantic plan and ports	ragjobs	ragjobs
semantic cache/artifacts/gates	ragjobs	ragjobs
queue maintenance	custom	River-provided
transaction enqueue	custom SQL	River client support
physical job UI/ecosystem	custom	River ecosystem
transport dependency	none beyond PostgreSQL	River schema/API
risk of dual retry	absent	must enforce one authority
workflow Pro features	not applicable	optional, not semantic source

17.6 Recommendation

For an organization already standardized on River and PostgreSQL, use River Core as the physical transport and retain ragjobs as the control/semantic layer. This reduces queue maintenance while keeping artifacts, source revisions, evaluation gates, and formal plan identity independent.

For an organization that needs minimal dependencies or unusually tight admission control, complete the direct PostgreSQL Store and conformance-test it against the memory interpreter.

Do not build an unrelated bespoke broker. PostgreSQL already participates in the control transaction, and the workload benefits more from correct state transitions than from extreme message throughput.

18. TTC integration

18.1 Existing TTC flow

The TTC workspace index command already contains the essential semantic body: experiment creation, workspace corpus preparation, resource declaration, provider retry through its flow layer, bundle construction, and completion metrics. The integration task is to extract these operations into versioned services rather than rewrite them.

`ragkit/flow` remains inside an expensive node. Its content-addressed item cache and request-level retry reduce repeated embedding work. `ragjobs` provides outer durability, ownership, and process-level recovery.

18.2 Handler mapping

ragjobs node	TTC adapter
snapshot	resolve commit and build immutable workspace manifest
extract-corpus	existing workspace loading and canonicalization
chunk-corpus	existing chunk/representation preparation
build-lexical	exposed lexical phase or descriptor from combined builder
embed-dense	existing flow embedding group
assemble-bundle	<code>ragkit/rag/indexbundle.Build</code>
verify-bundle	manifest/index open plus deterministic queries
evaluate-candidate	workspace evaluation and experiment machinery
quality-gate	versioned metric policy
publish	compare-and-swap workspace alias

A first migration may collapse lexical, dense, and assembly into `ttc.build_bundle.v1`. This preserves correctness but reduces retry/cache granularity. Decomposition should follow clean semantic APIs, not force artificial file boundaries.

18.3 Triggering

Repository changes resolve a branch or webhook ref to a commit SHA before run creation. The deduplication key combines commit, configuration digest, and target corpus. Rapid commits may be debounced to the newest commit when intermediate indexes have no business value.

TTC application-database changes use an outbox. Scheduled full verification is a durable periodic trigger that creates a run even when the source commit is unchanged, with cache modes chosen to permit structural re-verification without rebuilding deterministic artifacts.

18.4 Experiments

The scheduler creates the durable run first. The TTC handler creates or links an experiment using run and node identifiers. Experiment completion should occur before node completion so a successful node always points to committed evidence. If experiment persistence is in another database, it needs an idempotency key and reconciliation.

Evaluation arms can be represented as child plan nodes or as domain-level work inside an evaluation shard. The plan should expose the level at which independent retry and cost accounting matter.

18.5 Publication

TTC's existing content-addressed bundle construction is retained. The active workspace alias is separate from the bundle directory. Serving processes resolve the alias, open the immutable bundle, run a local health check, and then switch handles.

Shadow rollout runs scheduler builds while the existing direct command remains authoritative. Bundle manifests, digests, retrieval outputs, and evaluation metrics are compared before enabling manual and then automatic promotion.

19. GEC/CoinVault integration

19.1 Existing GEC boundaries

The supplied command group exposes useful boundaries despite the missing `internal/knowledgebuild` source. `knowledge build` calls the absent builder after loading MySQL and curated SQL-document sources. `knowledge inspect` validates/open bundles. `knowledge eval` evaluates reviewed questions over lexical and hybrid routes.

The scheduler adapter should be implemented where `knowledgebuild.Build` is restored or replaced. The CLI becomes a local/submission wrapper over the same handler services.

19.2 MySQL outbox

Every index-relevant product, category, synonym, curation, or domain mutation writes an outbox row in the same InnoDB transaction. Events identify GEC, corpus, entity, operation, stream revision, and optional non-sensitive payload.

The relay writes PostgreSQL inbox rows at least once. The coalescer adopts a contiguous prefix. The snapshot handler begins a consistent MySQL read and records the actual revision. This protocol avoids a distributed transaction while preserving eventual complete indexing.

Triggers should describe what changed but not carry the entire record as source truth. The snapshot reads authoritative rows. Payloads may help prioritize or choose an incremental future path, but a missed payload field cannot change correctness.

19.3 Handler mapping

ragjobs node	GEC adapter
snapshot	consistent MySQL snapshot plus curated-doc version
extract-corpus	product/category/document normalization from builder boundary
chunk-corpus	manifest-selected heading-aware chunking
build-lexical	existing lexical index construction
embed-dense	optional vector representation with pinned model
assemble-bundle	<code>ragkit/rag/indexbundle.Build</code> under bundle storage
verify-bundle	existing <code>inspect/open</code> behavior plus smoke queries
evaluate-candidate	existing <code>knowledge.RunEval</code> , sharded by case/route
quality-gate	lexical floors and hybrid comparison policy
publish	monotone CAS of <code>gec/knowledge/production</code> alias

The quality policy can require lexical retrieval to remain above a hard floor even when hybrid improves average quality. Serving configuration determines whether hybrid is enabled for the published generation.

19.4 K3s and GitOps deployment

A practical GEC deployment uses separate Kubernetes deployments for control/relay, CPU indexing, embedding, evaluation, and publication queues. Queue selection prevents embedding saturation from blocking control operations. Resource requests and limits reflect the stage.

Credentials are mounted through secret references or workload identity. Plans contain secret names, not secret values. Network policy permits workers only the source, artifact, provider, and control endpoints they require.

Serving pods either resolve object-store aliases or mount a shared read-only bundle store. They retain the previous opened bundle until the new generation passes local open and health checks. A rollout controller reports generation adoption so the control plane can detect pods stuck on an old bundle.

Two Production Interpreters, One Semantic Kernel

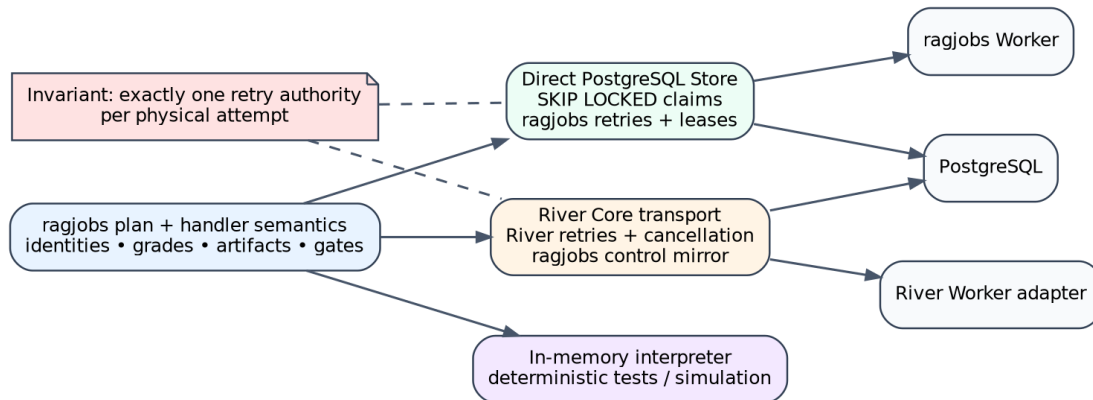


Figure 7. Suggested deployment topology with source outbox relay, PostgreSQL control plane, specialized worker pools, immutable artifact storage, and serving replicas.

19.5 Backfill and schema migration

A GEC schema or curation-policy change creates a new configuration epoch and explicit backfill run. The backfill may consume the current source revision but must not reuse snapshot semantics from an incompatible extractor. Old active bundles remain available throughout evaluation.

Large backfills should use a separate queue and budget so ordinary change latency remains bounded. Publication remains serialized with ordinary runs and subject to revision/policy checks.

19.6 Missing-package limitation

Because `internal/knowledgebuild` is absent from the supplied archive, no claim is made that the GEC adapter compiles against the original code. The report and integration guide specify the contract, SQL outbox, plan mapping, and deployment. Restoring or locating the package is a prerequisite to a concrete application patch.

20. Observability, operations, and security

20.1 Event model

Every meaningful transition appends a structured event with run, optional node, attempt, type, time, worker, class, message, and JSON data. Events are durable audit facts, not the only state representation. Current snapshots are materialized in run/node rows for efficient queries.

Recommended event data includes plan ID, fence, lease expiration, retry time, output digest, semantic key, cost, publication generation, source revision, cache provenance, and administrative actor. Messages are sanitized and bounded.

20.2 Metrics

Core metrics include:

- runs created, succeeded, failed, canceled by plan/system/corpus;
- node attempts and terminal failures by kind/version/class;
- ready age and queue latency by queue;
- handler duration and end-to-end change-to-publication latency;
- retry count and lease recoveries;
- cost consumed and reserved;
- cache hit rate and conflict count;
- artifact bytes written and verification failures;
- outbox/inbox revision lag and oldest pending age;
- candidate gate pass/reject rate;
- alias generation and serving adoption lag.

Metrics labels must be bounded. Run IDs, node IDs for generated shards, entity keys, digests, and error messages belong in traces/logs, not high-cardinality metric labels.

20.3 Tracing

A run is a trace or trace-linked workflow root. Each attempt is a span with run/node/attempt/fence attributes. Provider and database calls are child spans. Because a run may last hours and cross process restarts, span links are often more appropriate than one continuously open parent span. OpenTelemetry provides the instrumentation model ([OpenTelemetry Authors 2026](#)).

Events and traces cross-link by run and attempt identity. The durable event log remains available after trace sampling or retention expires.

20.4 Logs

Structured logs include run, plan, node, kind/version, attempt, fence, worker, queue, semantic key prefix, and artifact digest prefix. Secrets, full source rows, prompts containing sensitive data, and provider credentials are excluded.

A worker logs lease loss at informational or warning level depending on frequency; the rejected stale completion is a safety success, not necessarily a system failure. Repeated lease loss for healthy long tasks indicates heartbeat or duration misconfiguration.

20.5 Service-level objectives

Useful SLOs are tied to business freshness and safety:

- 99% of contiguous source revisions adopted into a run within a target time;
- 99% of eligible changes published or explicitly rejected by policy within a target time;
- zero unverified or policy-rejected automatic publications;
- zero accepted alias revision regressions;
- bounded oldest-ready age per queue;
- bounded lease-recovery rate;
- successful rollback drill within a target time.

A queue-depth SLO alone is insufficient because one expensive old job may matter more than thousands of fresh low-priority jobs.

20.6 Alerts

Page-worthy conditions include active artifact corruption, semantic cache conflict, alias revision regression attempt, inability to publish a verified emergency fix, control-database unavailability beyond threshold, growing contiguous outbox gap, and systematic gate failure after a release.

Ticket-level conditions include isolated permanent data errors, elevated retry rate, cache hit deterioration, storage growth, and long evaluation latency.

Alerts link to the run, source revision, plan, candidate digest, active alias generation, and relevant runbook section.

20.7 Administrative controls

Authenticated controls include cancel run, create rerun, create retry-from-node derivation, pause/drain queue at transport, manual promote, privileged rollback, invalidate cache, quarantine artifact, and release quarantine. Every action records actor, reason, previous state, requested state, and correlation/incident identifier.

Destructive actions use two-person approval where business risk warrants it. Automatic systems use service identities with narrower permissions than human operators.

20.8 Secret management

Plans, arguments, events, effect grades, semantic keys, and artifacts may persist for years. They must not contain credential values. A node argument contains a secret-manager reference or provider profile name. The worker resolves it at attempt time under workload identity.

Changing a credential does not change semantic identity. Changing a provider endpoint, model, or account behavior that affects values does, and must be represented separately in configuration.

20.9 Tenant and corpus isolation

Multi-tenant systems include tenant/corpus in run labels, source stream, artifact namespace, alias name, concurrency key, and authorization checks. A semantic cache may be shared only when data classification and encryption policy permit it. The key itself does not prevent a caller from learning that another tenant produced a digest.

PostgreSQL row-level security can supplement application authorization, but worker roles generally need queue-wide access. Separate schemas or databases may be appropriate for strict isolation.

20.10 Supply chain and handler registration

Workers report build version and supported handler protocols. Deployment must ensure at least one compatible worker remains for nonterminal old plans. An admission check can reject run creation when no worker pool advertises a required `(kind, version)`.

Artifacts include builder and dependency versions. Container images are pinned by digest. Database migrations are versioned and applied before workers requiring them start.

20.11 Incident procedures

A bad candidate is preserved, not deleted. The active alias remains unchanged. Operators inspect evaluation evidence, source/config fingerprints, handler versions, and cache provenance.

A bad publication is rolled back by an audited alias generation. The suspect digest is quarantined, automatic promotion pauses for the corpus, and serving adoption is verified.

During a control-database outage, source outboxes continue accumulating. Workers stop acquiring authority and leases expire. After recovery, operators verify database time, run lease recovery, resume relays, and monitor contiguous lag. Outbox rows are not discarded to make dashboards green.

21. Performance and capacity planning

21.1 Workload decomposition

Capacity should be modeled by resource queue, not by total node count. The reference plan uses control, CPU indexing, embedding, evaluation, and publication queues. Each has different service demand, concurrency limits, and cost.

Let $(_r)$ be indexing runs per second after coalescing. For queue (q) , let (D_q) be average worker-seconds demanded by one run and (c_q) effective parallel worker slots. A first utilization estimate is

$$[_q = _r \cdot D_q / c_q]$$

Stable low-latency operation requires $(_q < 1)$, with substantial headroom for variance, retries, backfills, and provider throttling. The formula is intentionally simple; actual service distributions for embeddings and evaluation are heavy-tailed.

For a sharded evaluation plan with (N) shards, per-run demand is the sum of shard demand, while critical-path duration is approximately the maximum shard duration plus enumeration and aggregation. Increasing (N) reduces critical path until queueing, fixed overhead, provider limits, or small-shard inefficiency dominates.

21.2 Change coalescing

Suppose source changes arrive at rate $(_c)$ and a debounce window of length (W) groups a contiguous prefix. Under a simple Poisson approximation, expected changes per nonempty window are related to $(_c W)$. The goal is not to maximize batch size blindly. Larger windows reduce rebuild frequency but increase freshness latency and the cost of a failed large candidate.

A useful adaptive policy considers:

- oldest unadopted change age;
- number and type of changed entities;
- current active build stage;
- estimated rebuild cost;
- business freshness target;
- whether an urgent/manual trigger bypasses debounce.

The coalescer must preserve contiguous revision semantics under every policy.

21.3 Full-rebuild cost model

Let:

- (n) be documents;
- (m) be chunks;
- (b) be embedding batch size;
- (p) be provider requests in flight;
- (t_e) average embedding request time;
- $(t_l(m))$ lexical build time;
- $(t_a(m))$ assembly/verification time.

Ignoring cache hits and throttling, dense critical time is approximately

$$[T_e,]$$

subject to provider request/token rate limits. Total build critical path is roughly

[T_{build} T_{snapshot} + T_{extract} + T_{chunk} + (T_{etl} (m)) + T_{a} (m) + T_{eval} + T_{publish} .]

Semantic reuse can reduce one or more terms to cache lookup and artifact verification. The model should therefore be measured separately for cold, warm-source-change, configuration-only, and evaluation-only runs.

21.4 Provider rate limits

Embedding and judge worker concurrency must respect both request and token quotas. A distributed token bucket or provider-aware limiter belongs inside the handler/provider client, because the scheduler's node concurrency is too coarse. The node effect grade and run budget remain the durable admission envelope.

Rate-limited attempts should not occupy CPU worker slots while sleeping for long periods. In direct mode the node returns to available with a future time. In River mode the worker returns the appropriate retry/snooze signal. The provider limiter exposes saturation so autoscaling does not add workers that only increase 429 responses.

21.5 Queue priorities and fairness

Priorities are useful for urgent production fixes, but unbounded priority can starve routine freshness. Candidate ordering should combine priority, availability time, run age, and stable identifiers, as the reference memory Store does.

Separate queues provide stronger isolation:

- control and publication remain responsive;
- ordinary indexing cannot be blocked by a massive evaluation;
- backfills use lower-priority dedicated capacity;
- provider-facing work scales independently of CPU work.

Fair-share scheduling by corpus or tenant can be added at claim time. It should be deterministic and preserve the Store's claim preconditions.

21.6 Backpressure

Backpressure begins at run creation, not only at worker pools. When change arrival exceeds sustainable rebuild rate, the system should coalesce more aggressively, supersede stale nonessential candidates, cap concurrent builds per corpus, and expose freshness degradation.

It should not create millions of nearly identical runs and hope the queue catches up. Deduplication and corpus-scoped active-run policy are control-plane responsibilities.

Evaluation can be sampled or deferred for low-risk changes only under an explicit policy. Automatic publication must still satisfy whatever evidence the policy requires.

21.7 Artifact storage growth

Let (S_b) be average bundle size, (S_i) average intermediate size, (r) builds per day, and (h) retained days. A rough storage bound before deduplication is

[$S r h (S_b + S_i)$.]

Content addressing deduplicates byte-identical objects but should not be assumed to deduplicate vector indexes across small corpus changes. Retention and future segment reuse matter more.

Capacity planning includes temporary peak space for concurrent branches and assembly. A worker must fail admission before disk exhaustion, not halfway through publication. The snapshot or plan can declare estimated bytes as a future effect grade.

21.8 Database load

Control-plane load is driven by attempts and heartbeats, not artifact bytes. If (a) attempts run concurrently and heartbeat every (h) seconds, steady heartbeat write rate is approximately (a/h). Event retention and indexing determine write amplification.

Heartbeat events may be sampled or stored in a separate lower-retention table at very high scale, while lease updates remain authoritative. The current reference records each heartbeat for clarity.

Claim queries use partial indexes over available rows. Old terminal nodes should not bloat the hot index. Autovacuum, fill factor, partitioning, and archival are operational concerns to test under realistic churn.

21.9 Autoscaling signals

CPU workers scale on ready age and CPU saturation. Embedding/evaluation workers scale on ready age subject to provider quota headroom. Publication workers normally need low fixed concurrency because aliases are serialized. Relay/coalescer capacity scales on revision lag.

Scaling on queue length alone can oscillate when many jobs are delayed by `available_at` or rate limits. Metrics distinguish ready-now, scheduled-future, running, and blocked work.

21.10 Benchmark plan

Before production, benchmark:

1. run creation with large plans;
2. claim throughput under many workers and queues;
3. heartbeat and lease-recovery load;
4. completion fan-out that makes many children ready;
5. semantic-cache contention on identical work;
6. event queries and archival;
7. outbox relay with duplicates and gaps;
8. alias contention and rollback;
9. cold and warm indexing on representative TTC/GEC corpora;
10. evaluation shard-size curves;
11. crash injection at every transition boundary.

Results should be recorded by schema version, database configuration, hardware, corpus size, and worker build. No numerical production capacity claim is made without those measurements.

22. Reference implementation

22.1 Module structure

The accompanying `ragjobs` module targets Go 1.23 and uses only the standard library. Its packages are:

Package	Responsibility
<code>job</code>	plan schema, composition, canonical identity, port validation, effects, states, events
<code>runtime</code>	Store interface, worker interpreter, registry, error classification, deterministic retry
<code>store/memory</code>	transactional in-process executable specification
<code>artifact</code>	SHA-256 file objects, verification, fenced/monotone alias CAS
<code>trigger</code>	normalized changes, gap splitting, contiguous prefix batching
<code>ragplan</code>	indexing and sharded evaluation plan constructors
<code>cmd/ragjobs-demo</code>	end-to-end execution with intentional rate-limit retry
<code>migrations</code>	PostgreSQL control plane and source outbox schemas
<code>integrations</code>	River, TTC, and GEC adoption contracts

The module is intentionally independent of TTC, GEC, `ragkit`, River, and a PostgreSQL driver. Application adapters import it and register handlers. A PostgreSQL adapter can accept `*sql.DB` from any chosen driver without changing semantic packages.

22.2 Plan types

`NodeSpec` contains stable ID, kind, version, queue, priority, canonical arguments, dependencies, typed ports, retry, timeout, cache mode, failure mode, effect grade, and labels. `Plan` contains schema version, plan ID, name, version, nodes, and labels.

`RunInput` contains trigger, values, labels, and budget. The trigger separates source cursor and deduplication key. The cursor is semantic; the deduplication key is operational.

`Finalize` clones the plan, applies defaults, sorts canonical sets, validates, and computes identity. It is referentially transparent from the caller's perspective.

22.3 Composition API

The compositional API is small:

```

snapshot := job.Atom(snapshotSpec)
extract  := job.Atom(extractSpec)
lexical  := job.Atom(lexicalSpec)
dense    := job.Atom(denseSpec)

prefix, _ := job.Then(snapshot, extract)
branches, _ := job.Tensor(lexical, dense)
flow, _ := job.Then(prefix, branches)
plan, _ := flow.Plan("rag-indexing", "v1", labels)

```

Then connects every right entry to every left exit. Tensor rejects identifier collisions and canonicalizes disjoint union. Plan finalization performs direct port compatibility checks.

The API is deliberately data-oriented. Go generic type parameters could improve compile-time application code but would not replace runtime validation of persisted JSON plans. A future builder can add typed generic wrappers over the same serialized core.

22.4 Store contract

The runtime.Store interface is the operational semantics:

```
type Store interface {
    CreateRun(context.Context, CreateRunRequest) (CreateRunResult, error)
    Claim(context.Context, ClaimRequest) ([]Lease, error)
    Heartbeat(context.Context, Lease, time.Time, time.Duration) error
    Complete(context.Context, Completion) error
    Fail(context.Context, Failure) error
    RecoverExpired(context.Context, time.Time, int) (int, error)
    CancelRun(context.Context, string, string, time.Time) error
    LookupCached(context.Context, string) (CachedResult, bool, error)
    GetRun(context.Context, string) (job.RunSnapshot, error)
    Events(context.Context, string, int64, int) ([]job.Event, error)
    ListRuns(context.Context, ListFilter) ([]job.RunSnapshot, error)
}
```

A durable implementation should make each mutating method one transaction. The interface returns semantic leases rather than raw queue rows. This allows the worker to remain transport-neutral.

22.5 Memory Store

The memory Store uses a mutex around each method and clones values at boundaries. It maintains run/node maps, child adjacency, event sequence, cache, and trigger deduplication. Claim ordering is priority descending, availability ascending, run creation ascending, then stable identifiers.

The Store implements cost reservation, global concurrency keys, fenced heartbeat/completion/failure, lease recovery, cancellation, skip propagation, fail-run and continue-run behavior, and terminal finalization. It is suitable for tests, local execution, and differential conformance. It is not a multi-process production store.

22.6 Worker interpreter

A worker polls accepted queues, receives leases, and executes them with bounded concurrency. For a cacheable node it computes the semantic key and attempts fenced reuse. Otherwise it resolves a handler by kind/version.

The worker creates an attempt timeout, runs a heartbeat goroutine when the lease is long enough, recovers panics, validates JSON output, computes its digest, and submits completion. Handler errors are classified and converted into a durable failure with retry time. Parent shutdown leaves a canceled in-flight handler unresolved when appropriate so lease recovery, rather than a domain cancellation, preserves at-least-once behavior.

Heartbeat errors are not used to kill arbitrary handlers because cancellation may be ineffective. Final completion still requires the current fence.

22.7 Handler registry

The registry maps (kind,version) to a Handler. Duplicate registration is rejected. Invocation contains run, plan, node specification, run input, direct dependencies, attempt, fence, and worker.

A handler returns JSON output, measured cost, and error. Domain adapters should return typed envelopes and classified errors. They should not update run/node state directly.

22.8 Artifact implementation

`artifact.FileStore` creates object and alias namespaces. Put streams with a context-aware reader, hashes, synchronizes, and renames. Existing objects are verified. `Open` and `Verify` enforce digest form and integrity.

Alias publication uses file locking, expected-digest compare-and-swap, idempotent same-digest/same-revision replay, monotone revision rejection, generation increment, provenance, temporary write, sync, and rename. The implementation is appropriate for a local/shared filesystem with reliable locks; PostgreSQL or object-store conditions are preferred across distributed filesystems.

22.9 Trigger implementation

`trigger.Change` is the normalized relay record. `Coalesce` is replay-stable and splits groups at revision gaps. `CoalesceContiguous` accepts a durable committed cursor map and emits only the next complete prefix. `Batch.Trigger` produces a source cursor and deterministic deduplication key.

The package does not itself persist the committed coalescer cursor. The PostgreSQL control service must update that cursor transactionally with run adoption or use equivalent inbox claims and run deduplication.

22.10 RAG plans

`ragplan.IndexingPlan` validates required system, corpus, cursor, configuration hash, and alias. It creates explicit queues, timeouts, retry policies, typed ports, effect grades, and publication key. Options control immutable-source snapshot caching, evaluation, evaluation caching, and quality gating.

`ragplan.EvaluationPlan` creates enumeration, stable shard nodes, aggregation, and optional comparison. Shards default to non-cacheable and can be explicitly declared cacheable under a deterministic evaluator contract.

22.11 Demonstration

The demo registers simple handlers for every indexing stage, stores artifacts in a temporary content-addressed store, and intentionally returns a rate-limited error on the first dense-embedding attempt. A deterministic clock advances to the retry time. The run then assembles, verifies, evaluates, gates, publishes, and cleans up.

The observed run succeeds with twelve attempts across eleven nodes. `embed-dense` consumes two attempts; every other node consumes one. The run records 1,500,000 cost microunits and a complete 38-event trace. The exact temporary directory changes by execution; plan and semantic behavior remain stable.

22.12 Database contracts

`migrations/postgres/0001_ragjobs.sql` defines the control schema. `0002_helpers.sql` provides event append and a fenced `SKIP LOCKED` claim function. `0003_source_outbox.sql` provides an optional PostgreSQL source outbox. The MySQL migration provides GEC source revision and outbox tables.

These migrations are reviewed design artifacts, not a substitute for live integration tests, migration rollback planning, or application-specific schema qualification.

23. Verification and validation

23.1 Validation environment

The completed reference module was validated on 7 August 2026 with Go 1.23.2 in the supplied execution environment. The validation commands were:

```
gofmt -l .
go vet ./...
go test ./... -count=1
go test -race ./... -count=1
go run ./cmd/ragjobs-demo
```

gofmt -l produced no files. go vet passed. Ordinary and race-enabled tests passed. The demo completed successfully after its intentional transient failure.

23.2 Test coverage by property

The distribution contains twenty-five named tests at the time of this report.

Property	Representative test
sequential associativity	TestSequentialCompositionAssociative
identity and tensor symmetry	TestIdentityAndTensorSymmetry
boundary dependency construction	TestThenAddsOnlyBoundaryDependencies
canonical plan identity and purity	TestPlanIdentityCanonicalAndFinalizePure
semantic input projection	TestSemanticKeyIgnoresOperationalMetadataButIncludesCursor
cycle rejection	TestValidationRejectsCycle
effect aggregation	TestAggregateEffects
direct artifact-port compatibility	TestValidationChecksDirectPortTypes
retry, dependency, event sequence	TestRetryDependencyAndEventTrace
cross-run semantic reuse	TestSemanticCacheReuseAcrossEquivalentTriggers
lease recovery and stale fencing	TestLeaseRecoveryFencesStaleCompletion
cancellation fencing	TestCancellationFencesWorker
budget and concurrency key	TestBudgetAndConcurrencyKey
continue-run branch behavior	TestContinueRunPreservesIndependentBranch
conflicting semantic results	TestConcurrentSemanticConflictIsRejected
trigger dedup and cost-grade enforcement	TestRunDedupAndCostGrade
artifact content addressing and alias CAS	TestContentAddressingAndAliasCAS
corruption and traversal defense	TestVerifyDetectsCorruptionAndRejectsTraversal
monotone alias revision and idempotent replay	TestAliasRejectsOlderRevisionAndAcceptsIdempotentReplay
deterministic change coalescing	TestCoalesceDeterministic
gap splitting and contiguous prefix	TestCoalesceSplitsGapsAndContiguousPrefixWaits

Property	Representative test
indexing plan shape/safety	TestIndexingPlanShapeAndSafety
gate/evaluation dependency	TestQualityGateRequiresEvaluation
evaluation fan-out/fan-in	TestEvaluationPlanShards
typed terminal publication input	TestIndexingPublishConsumesTerminalCandidate

Tests target invariants rather than line coverage alone. The race run exercises concurrent worker/store behavior in the test scenarios.

23.3 Evidence limits

Passing tests do not prove the PostgreSQL schema and future adapter preserve every Store transition. No live PostgreSQL server was available, so SQL functions were not executed under concurrency, crash, or migration tests. No River dependency was installed, so the adapter remains a protocol and integration guide rather than compiled code.

The original TTC/GEC modules were not rebuilt because their declared Go versions and external dependencies exceed the offline environment. GEC's builder package is absent. The reference module is independently compiled.

The formal arguments are paper proofs and executable examples. They have not been mechanized in Lean, Coq, Isabelle, TLA+, Alloy, or a model checker.

23.4 Required production tests

Before automatic publication, add:

1. PostgreSQL Store conformance tests against the memory Store;
2. multi-process claim contention tests;
3. kill-at-every-transition crash tests;
4. database failover and clock tests;
5. River duplicate, cancellation, and transactional-completion tests if used;
6. outbox relay loss, duplicate, reorder, and gap tests against MySQL/PostgreSQL;
7. object-store conditional-write and corruption tests;
8. TTC/GEC golden-corpus equivalence tests;
9. load tests with realistic corpus and provider quotas;
10. security tests for authorization, secret leakage, path/payload limits, and tenant isolation.

23.5 Differential state-machine testing

A high-value next step is command-sequence testing. Generate valid and invalid operations—create, claim, heartbeat, complete, fail, recover, cancel—and execute them against the memory and PostgreSQL stores with a controlled clock. Normalize timestamps and database identifiers, then compare snapshots, events, errors, cache, and budget state.

This directly tests that the SQL interpreter refines the executable specification. Random crash points can roll back or commit transactions and resume from persisted state.

23.6 Property and model checking

Property-based tests can generate acyclic plans and verify composition, readiness, terminality, and schedule-independence under pure handlers. A small TLA+ model can explore duplicate delivery, lease expiry, cancellation, and concurrent completion. A proof assistant can formalize the graded computation and open-DAG category.

Mechanization should focus first on the operational Store invariants because they carry the highest production risk. The categorical laws are comparatively simple in the implementation.

24. Deployment and adoption plan

24.1 Phase 0: protocol stabilization

Freeze plan schema version 1, event vocabulary, handler versioning rules, artifact envelope, source revision semantics, and alias protocol. Review the model with TTC, GEC, database, platform, security, and evaluation owners.

Select direct PostgreSQL or River transport. Do not implement both initially. Define one retry authority and one operational owner.

24.2 Phase 1: PostgreSQL control service

Implement the durable Store or River control adapter. Apply schemas in a nonproduction database. Add conformance and crash tests. Build read APIs and a minimal operator UI/CLI for run, node, attempts, events, cancellation, and candidate promotion.

Deploy workers with no production handlers or with synthetic handlers. Validate leases, recovery, budgets, retention, backup, and alerting.

24.3 Phase 2: handler extraction

Refactor TTC command bodies into versioned services and register handlers. Restore or implement GEC's knowledge-builder service boundary. Keep existing CLI commands able to execute handlers locally for development.

Add artifact envelopes and source/config fingerprints. Pin external model and analyzer versions. Classify errors explicitly.

24.4 Phase 3: source triggers

For TTC repository sources, implement commit resolution, webhook/poll reconciliation, and deduplication. For GEC, deploy MySQL outbox writes, relay, PostgreSQL inbox, durable contiguous cursor, and coalescer.

Run relay shadow mode first: compare observed outbox revisions with domain mutations without creating builds. Test gaps and replay.

24.5 Phase 4: shadow builds

Create scheduler runs for real changes but do not publish. Compare candidate manifests, bundle digests where deterministic, retrieval outputs, costs, and durations against the existing build path.

Investigate every semantic-cache conflict and artifact mismatch. Establish cold/warm baselines and queue capacity. Keep candidates for forensic comparison.

24.6 Phase 5: evaluation and manual promotion

Enable deterministic verification and evaluation. Define versioned quality policy with domain owners. Require manual promotion through the alias protocol. Practice rollback and artifact quarantine.

Serving processes report loaded generation and health. Verify that a failed candidate cannot change the active alias.

24.7 Phase 6: automatic gated promotion

Enable automatic publication for low-risk change classes after a sustained shadow/manual period. Retain manual approval for schema, model, chunker, or policy epochs until sufficient evidence accumulates.

Set SLOs and alerts for freshness, gate failures, revision lag, lease recovery, and serving adoption. Conduct failure drills.

24.8 Phase 7: optimization

Only after correctness and operations stabilize, add adaptive coalescing, partial rerun, immutable segments, compaction plans, shared cross-corpus caches where authorized, and advanced scheduler fairness.

Optimization must preserve plan identity, artifact immutability, evaluation evidence, and publication semantics.

24.9 Schema migration discipline

Database migrations are backward-compatible across rolling deployments. Add columns/tables first, deploy code that writes both representations if necessary, backfill, switch readers, and remove old fields in a later release.

Handler protocol versions and plan schema versions are separate. A database migration should not silently reinterpret old plan JSON.

24.10 Backup and disaster recovery

Back up PostgreSQL control state and alias rows with point-in-time recovery. Artifact storage has independent durability and inventory. Source outboxes remain the replay origin for unadopted changes.

A disaster-recovery exercise restores control data, verifies alias objects/digests, restarts relays from durable cursors, recovers expired leases, and reconciles physical River jobs if applicable. If control history is lost but artifacts survive, aliases and source revision state need a carefully audited reconstruction; this is a degraded procedure, not normal operation.

24.11 Multi-region considerations

A single writer region for one alias simplifies ordering. Workers may run near providers or source replicas, but completion and publication authority return to the writer control plane. Cross-region database latency affects heartbeats and should be measured.

Active-active alias writers require a globally linearizable compare-and-swap and one source revision order. Without that, use region-specific candidates and a separate global promotion service.

24.12 Production readiness checklist

Automatic publication remains disabled until all of the following are true:

- durable Store or River adapter passes conformance and crash tests;
- source outbox/commit trigger proves no-loss and contiguous behavior;
- every handler has version, timeout, retry classification, cache decision, and cost grade;
- immutable artifacts verify and garbage collection has a dry-run mode;
- quality policy and baseline resolution are versioned;
- alias CAS, monotone revision, idempotent replay, manual promotion, and rollback are tested;
- serving generation adoption is observable;
- cancellation and lease recovery are exercised;
- secrets and tenant boundaries are reviewed;
- retention, backup, disaster recovery, and incident runbooks are approved;
- shadow builds show acceptable correctness, latency, and cost.

25. Limitations and future work

25.1 No live durable adapter in this environment

The largest implementation limitation is the absence of a live-tested PostgreSQL Store or compiled River adapter. The schemas and transaction contracts are detailed, and the memory Store supplies a conformance target, but production deployment requires writing and testing the concrete adapter.

A thin database/sql adapter is feasible without coupling semantic packages to a driver. It should not be rushed without a PostgreSQL test matrix because completion, propagation, cache contention, and recovery are correctness-critical.

25.2 Port schemas are nominal

Plan finalization checks nominal port type names across direct dependencies. It does not yet fetch JSON Schema, Protocol Buffers descriptors, or Go types and validate output bodies automatically. Artifact readers and handlers perform structural validation.

A future schema registry can version schemas, validate envelopes at completion, and generate typed handler wrappers. Backward/forward compatibility rules should be explicit.

25.3 Formalization is not mechanized

The category, graded computation, operational transitions, and proofs are stated precisely enough to guide implementation but remain on paper. Mechanizing the Store invariants would increase assurance, especially around cancellation, failure propagation, and River dispatch reconciliation.

TLA+ is a practical first step for finite-state transition exploration. A proof assistant is appropriate for the algebra and refinement theorem once the production protocol stabilizes.

25.4 External side effects remain application-specific

The framework identifies required patterns but cannot automatically make a provider or legacy database operation idempotent. Each handler must document its ambiguity protocol. A future effect capability interface could require handlers with `ExternalCalls` to declare idempotency or reconciliation methods.

25.5 Current effect grade is coarse

The grade tracks logical read/write sets, external classes, cost bound, and node-local concurrency key. It does not express expected bytes, memory, GPU, tokens, privacy class, locality, or compensations. Extending the grade into a richer ordered semiring or resource algebra could improve admission and schedule placement.

The extension should preserve simple serialization and avoid turning every plan into an undecidable static-analysis problem.

25.6 Snapshot and cache policy needs domain proof

The generic plan correctly disables cache for mutable current-state snapshots, but application adapters must still prove immutable-source claims and pin every dependency. A moving provider model or curated-document alias can invalidate semantic determinism without a code change.

A dependency attestation artifact could record all resolved versions and be included in semantic keys.

25.7 Evaluation governance

The scheduler can execute and preserve evaluation, but it cannot decide which metrics are ethically or commercially sufficient. Dataset quality, leakage, judge bias, statistical power, and domain risk require separate governance.

Future work can add signed evidence, policy-as-code review, approval workflows, and reproducibility attestations.

25.8 Partial rerun and derived runs

The current administrative model favors a new run and semantic reuse. A first-class derived-run operation could select a failed node, preserve the same plan/input or a documented override, and pre-adopt successful upstream outputs. The derived run would have its own identity and lineage to the parent.

This is safer than reopening terminal nodes and can support operator repair after a permanent external defect is corrected.

25.9 Dynamic fan-out

The static evaluation plan creates a fixed number of shard nodes. Some workloads need dynamic fan-out based on enumerated items. A production extension can let an enumeration node transactionally materialize a validated subplan whose identity is derived from the parent plan and enumeration digest.

Dynamic expansion needs explicit limits, deterministic node naming, schema validation, and a proof that replay creates the same subplan.

25.10 Segment-level incremental indexing

Immutable segments and tombstone manifests are the natural performance evolution. Research questions include optimal compaction scheduling, cross-index consistency, segment-level semantic keys, evaluation sampling for small deltas, and proof that a published manifest represents one source revision.

The existing artifact/alias semantics provide a compatible foundation.

25.11 Priority, fairness, and admission

The reference Store uses deterministic priority/age ordering and global concurrency keys. Multi-tenant production may require weighted fair queues, per-corpus active-run limits, reservation quotas, and deadline-aware scheduling.

These policies should refine claim admissibility without changing node causality or fenced completion.

25.12 Privacy and deletion

Content-addressed immutable artifacts complicate legal deletion. A production system needs data-classification metadata, encryption-key scoping, tombstone and purge workflows, and evidence that all reachable and cached copies of a subject's data are removed when required.

One approach encrypts tenant/corpus artifacts with rotatable keys so cryptographic erasure complements physical garbage collection. This is outside the current reference implementation.

26. Conclusion

Production RAG indexing is a durable derivation and publication problem, not merely a background function call. The source changes asynchronously; the work is expensive; execution is at least once; external effects are not transactional; and the active retrieval state must remain valid throughout failure.

The proposed system addresses this by separating semantic plans from physical queue delivery. Versioned nodes compose into canonical dependency graphs. Graded Kleisli semantics explains value transformation with state, failure, trace, cost, and controlled nondeterminism. A structural operational semantics defines durable claims, leases, fences, retries, recovery, cancellation, and finalization. Immutable artifacts and semantic identities make deterministic work reusable. A minimal compare-and-swap alias makes publication linearizable. Contiguous source adoption and monotone revisions prevent late changes from rolling the index backward.

The accompanying Go implementation demonstrates these ideas as executable code. It passes static, ordinary, race-enabled, and end-to-end validation in the available environment. PostgreSQL schemas and River contracts show how to carry the semantics into a multi-process deployment. TTC can wrap its existing workspace indexing and experiment logic. GEC can connect its MySQL domain through a transactional outbox and its knowledge commands through versioned handlers once the missing builder boundary is restored.

The principal operational recommendation is direct: adopt immutable micro-batched bundles first; use River as transport when it reduces maintenance; keep one retry authority; treat cacheability as a semantic assertion; never publish without verification and explicit policy; and make every source revision, artifact digest, attempt, and alias generation explainable.

The result is not arbitrary exactly-once execution. It is a compositional system whose actual guarantees are named, testable, and strong enough for production indexing and evaluation.

Appendix A. Core serialized contracts

A.1 Plan

A canonical plan has the conceptual JSON shape:

```
{
  "schema_version": 1,
  "plan_id": "rjp-...",
  "name": "rag-indexing",
  "version": "v1",
  "nodes": [
    {
      "id": "embed-dense",
      "kind": "rag.embed_dense",
      "version": "v1",
      "queue": "embedding",
      "priority": 0,
      "args": {"configuration_hash": "sha256:..."},
      "depends_on": ["chunk-corpus"],
      "inputs": [{"name": "chunks", "type": "rag/chunk-set"}],
      "outputs": [{"name": "vectors", "type": "rag/vector-index"}],
      "retry": {
        "max_attempts": 6,
        "base_delay": 10000000000,
        "max_delay": 900000000000,
        "multiplier": 2,
        "jitter": 0.2
      },
      "timeout": 14400000000000,
      "cache": "content",
      "failure_mode": "fail_run",
      "effects": {
        "reads": ["artifact-store"],
        "writes": ["artifact-store"],
        "external_calls": ["embedding-provider"],
        "cost_microunits_max": 100000000
      }
    }
  ],
  "labels": {"rag.system": "gec", "rag.corpus": "knowledge"}
}
```

Go `time.Duration` values serialize as integer nanoseconds in the current schema. A future language-neutral schema may encode ISO 8601 durations or explicit milliseconds; that requires a plan schema version.

A.2 Run input

```
{
  "trigger": {
    "kind": "db-change-batch",
    "source": "mysql/gec",
    "cursor": "1042",
    "dedup_key": "change-batch-...",
    "payload": {"from_revision": 1038, "to_revision": 1042},
    "occurred_at": "2026-08-07T12:00:00Z"
  },
  "values": {
    "configuration_digest": "sha256:...",
    "quality_policy_digest": "sha256:..."
  },
  "labels": {"environment": "production"},
  "budget": {
    "max_cost_microunits": 250000000,
    "max_total_attempts": 80,
    "deadline": "2026-08-08T12:00:00Z"
  }
}
```

```
}  
}
```

Only trigger kind/source/cursor/payload and values participate in the semantic node key. Dedup key, occurrence time, labels, and budget do not.

A.3 Handler output

The runtime requires valid JSON. Recommended output is a typed immutable envelope. The runtime computes a canonical JSON digest and stores the result. A handler reports actual cost separately so it cannot silently alter semantic output identity through billing metadata.

A.4 Events

Event types are:

```
run_created, run_started, run_succeeded, run_failed, run_canceled,  
node_ready, node_started, node_heartbeat, node_retry_scheduled,  
node_succeeded, node_failed, node_canceled, node_skipped,  
node_result_reused, lease_recovered
```

Production adapters may add versioned event data fields but should not reinterpret existing event types.

Appendix B. Operational transition table

Transition	Required state	Principal checks	Result
CREATE	run absent	plan valid; dedup absent	queued run; roots available
CLAIM	node available	time, queue, budgets, key	running node; attempt/fence incremented
HEARTBEAT	current running lease	owner, attempt, fence, unexpired	expiration extended
CACHE-HIT	current running lease	key exists; fenced completion valid	succeeded/reused
SUCCEED	current running lease	JSON/digest/cost/cache consistency	succeeded; children propagated
RETRY	current running lease	retryable; attempts/budgets remain	available at retry time
FAIL	current running lease	terminal condition	failed; cancel or skip propagation
RECOVER	expired running lease	expiration reached	lease-expired attempt; retry/fail
CANCEL	run nonterminal	authenticated request	remaining nodes and run canceled
FINALIZE	all nodes terminal	outcome classification	run succeeded or failed

Every transition appends one or more events in the same atomic Store method.

Appendix C. PostgreSQL transaction pseudocode

C.1 Completion

```

BEGIN;
SELECT run FOR UPDATE;
SELECT node FOR UPDATE;
assert run nonterminal;
assert node running;
assert owner, attempt, fence match;
assert lease_expires_at > database_now;
verify canonical output digest and declared cost;

if semantic_key present:
    SELECT cache row FOR UPDATE;
    if absent: INSERT cache;
    if digest differs: ROLLBACK with semantic conflict;

UPDATE attempt as succeeded;
UPDATE run release reservation, add actual cost;
UPDATE node succeeded, output/digest/key, clear lease;
append success/reuse event;

for each child that now has all dependencies succeeded:
    UPDATE child blocked -> available;
    append node_ready;

propagate skips if needed;
finalize run if all nodes terminal;
COMMIT;

```

C.2 Failure

```

BEGIN;
lock and validate current lease;
finish attempt; release reservation; add actual cost;

if class retryable and node/run budgets permit:
    set node available_at = retry_at, state = available;
    clear lease; append retry event;
else:
    set node failed; clear lease; append failed event;
    apply failure_mode;
    propagate skipped descendants;
    finalize run if terminal;
COMMIT;

```

C.3 Monotone alias publication

```

BEGIN;
SELECT alias FOR UPDATE;
if current revision == proposed revision and digest == proposed digest:
    return current alias (idempotent success);
if both revisions ordered and proposed <= current:
    reject stale revision;
if expected digest/generation does not match current:
    reject conflict;
verify candidate object and gate evidence;
UPDATE alias digest, generation+1, revision, provenance;
append publication event;
COMMIT;

```

A privileged rollback uses a separate audited procedure that can override monotone revision.

Appendix D. Operational runbook

D.1 Run stuck in running state

Check worker heartbeat, node lease expiration, control-database time, and reaper health. Do not manually clear owner fields. Run the fenced recovery operation. If the handler has an ambiguous external operation, reconcile it before allowing retry.

D.2 Node repeatedly retries

Inspect classified error, retry interval, provider status, attempt count, cost, and handler version. Confirm nested inner/outer retry is bounded. Pause the relevant queue or cancel the run if retries create harm. Convert misclassified permanent failures to a new handler version; do not mutate historical events.

D.3 Semantic cache conflict

Quarantine the node kind/version and key. Preserve both attempted outputs and source worlds if available. Check incomplete arguments, moving model aliases, mutable snapshots marked cacheable, nondeterministic serialization, locale/time dependencies, and corrupted artifacts. Do not auto-retry as transient.

D.4 Outbox gap

Identify missing revision and source transaction. Confirm relay has not skipped a pending row and target inbox dedup has not rejected a malformed duplicate. Keep later revisions unprocessed. If a source revision was legitimately not assigned due rollback, verify allocation transaction semantics before an audited cursor repair.

D.5 Candidate rejected by gate

Leave active alias unchanged. Inspect candidate and baseline evidence, slice metrics, policy digest, source/config changes, and evaluation validity. Create a corrected new run or an explicitly approved manual promotion; do not edit evidence.

D.6 Alias conflict or stale revision

Read current alias generation, digest, and revision. If it already equals the candidate, treat as idempotent success. If current revision is newer, mark old run superseded/stale. If an unrelated same/newer candidate won, rerun comparison or require operator decision. Never use unconditional replacement in an automatic publisher.

D.7 Bad active bundle

Stop automatic publication for the corpus. Advance the alias to the prior verified digest through privileged rollback. Verify serving adoption and local health. Quarantine the bad digest and preserve the incident run. Start a corrected build under a new version/configuration.

D.8 Scheduler database outage

Source outboxes continue. Workers cease obtaining authority; in-flight leases eventually expire. Restore database service and time correctness, run lease recovery, reconcile River dispatches if used, resume relays, and monitor oldest contiguous revision age. Do not delete backlog.

D.9 Provider ambiguous request

Use provider idempotency or operation ID to query status. If committed, adopt the immutable result under the current semantic input where permitted. If definitely absent, retry. If unknowable and non-idempotent, fail for manual reconciliation rather than issue a blind duplicate.

Appendix E. Validation evidence

The final validation log in the source distribution records:

```
go version go1.23.2 linux/amd64

gofmt -l .
(no output)

go vet ./...
PASS

go test ./... -count=1
(all packages passed)

go test -race ./... -count=1
(all packages passed)
```

The demonstration's terminal summary is:

```
state:      succeeded
attempts:   12
cost:       1500000 microunits
embed-dense: succeeded, attempts=2
all other ten nodes: succeeded, attempts=1
```

The first dense attempt emits `node_retry_scheduled` with class `rate_limited`; the second succeeds after deterministic clock advancement. The run then evaluates, gates, publishes, cleans up, and emits `run_succeeded` as event 38.

Appendix F. Glossary

Alias. Mutable named pointer to an immutable artifact digest, with generation and provenance.

Artifact. Immutable validated value or descriptor, normally content addressed.

Attempt. One physical execution ownership interval for a node.

Cacheability. Assertion that a node is observationally deterministic under its semantic key.

Committed cursor. Highest source revision durably adopted by the coalescer for a stream.

Concurrency key. Logical name that permits at most one running node globally under the Store.

Control plane. Mutable transactional state for runs, nodes, attempts, events, caches, triggers, and aliases.

Data plane. Immutable corpus, index, vector, bundle, and evaluation artifacts.

Deduplication key. Operational trigger identity used to suppress duplicate run creation.

Dispatch generation. River-mode counter identifying an intentional physical redispach for one semantic node.

Effect grade. Conservative summary of logical resources, external calls, and maximum cost.

Fence. Monotonically increasing token required for a worker to mutate current node state.

Handler. Versioned implementation of one atomic node kind.

Lease. Time-bounded fenced capability granting one worker authority over one node attempt.

Plan. Canonical immutable versioned dependency graph.

Publication. Conditional alias transition after verification and policy.

Run. One execution of a plan over immutable run input and source trigger.

Semantic key. Digest of node protocol, canonical arguments, semantic run input, and dependency digests.

Source revision. Monotone positive order value within one source/system/corpus stream.

Store. Transactional interpreter of the operational semantics.

Tensor. Parallel/disjoint composition of independent workflow fragments.

Then. Sequential composition that adds causal edges from left exits to right entries.

Appendix G. Distribution map

```
ragjobs/  
  README.md  
  go.mod  
  job/  
  runtime/  
  store/memory/  
  artifact/  
  trigger/  
  ragplan/  
  cmd/ragjobs-demo/  
  migrations/postgres/  
  migrations/mysql/  
  integrations/river/  
  integrations/ttc/  
  integrations/gec/  
  docs/  
    FORMAL_SPEC.md  
    POSTGRES_STORE.md  
    OPERATIONS.md  
  adr/  
  figures/  
  thesis.md  
  references.bib  
  validation.txt
```

The PDF and DOCX editions of this report are generated from docs/thesis.md. The source archive includes all code, tests, SQL, diagrams, and integration notes.

References

- Bernstein, Philip A., Vassos Hadzilacos, and Nathan Goodman. 1987. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley.
- Garcia-Molina, Hector, and Kenneth Salem. 1987. “Sagas.” In *Proceedings of SIGMOD*, 249–59. <https://doi.org/10.1145/38713.38742>.
- go-go-golems. 2026a. “GEC/CoinVault RAG Application.”
- . 2026b. “Ragkit: Reusable RAG Building Blocks for Go.”
- Gray, Jim, and Andreas Reuter. 1993. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
- Helland, Pat. 2007. “Life Beyond Distributed Transactions: An Apostate’s Opinion.” In *CIDR*, 132–41.
- Kahn, Gilles. 1974. “The Semantics of a Simple Language for Parallel Programming.” In *Information Processing 74*, 471–75. North-Holland.
- Katsumata, Shin-ya. 2014. “Parametric Effect Monads and Semantics of Effect Systems.” In *Proceedings of POPL*, 633–46. <https://doi.org/10.1145/2535838.2535846>.
- Mac Lane, Saunders. 1998. *Categories for the Working Mathematician*. 2nd ed. Springer. <https://doi.org/10.1007/978-1-4757-4721-8>.
- Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55–92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4).
- OpenTelemetry Authors. 2026. “OpenTelemetry Specification.” <https://opentelemetry.io/docs/specs/>.
- Orchard, Dominic, Philip Wadler, and Harley Eades. 2020. “Unifying Graded and Parameterised Monads.” *Proceedings of the ACM on Programming Languages* 4 (POPL). <https://doi.org/10.1145/3371116>.
- Plotkin, Gordon D. 1981. “A Structural Approach to Operational Semantics.” DAIMI FN-19. Aarhus University.
- PostgreSQL Global Development Group. 2026. “PostgreSQL 18 Documentation: SELECT and SKIP LOCKED.” <https://www.postgresql.org/docs/current/sql-select.html>.
- River Queue. 2026a. “Periodic and Cron Jobs.” <https://riverqueue.com/docs/periodic-jobs>.
- . 2026b. “River Pro Workflows.” <https://riverqueue.com/docs/pro/workflows>.
- . 2026c. “River: Fast and Reliable Background Jobs in Go.” <https://riverqueue.com/>.
- . 2026d. “Transactional Enqueueing.” <https://riverqueue.com/docs/transactional-enqueueing>.
- . 2026e. “Transactional Job Completion.” <https://riverqueue.com/docs/transactional-job-completion>.
- . 2026f. “Unique Jobs.” <https://riverqueue.com/docs/unique-jobs>.
- . 2026g. “Writing Reliable Workers.” <https://riverqueue.com/docs/reliable-workers>.
- The Tree Center. 2026. “Rag-Ttc Research Codebase.”
- Winskel, Glynn. 1987. “Event Structures.” In *Petri Nets: Applications and Relationships to Other Models of Concurrency*, 255:325–92. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/3-540-17906-2_31.